

# FINDS: The Fish Interaction and Dynamics Simulator

1.0

Generated by Doxygen 1.9.8

|                                                                                           |          |
|-------------------------------------------------------------------------------------------|----------|
| <b>1 FINDS: The Fish Interaction and Dynamics Simulator</b>                               | <b>2</b> |
| 1.1 SETUP                                                                                 | 2        |
| 1.1.1 Setup Example for Debian-Derived Systems                                            | 2        |
| 1.1.2 Docker                                                                              | 3        |
| 1.1.3 GNU Make                                                                            | 3        |
| 1.2 Acknowledgements                                                                      | 4        |
| <b>2 Hierarchical Index</b>                                                               | <b>4</b> |
| 2.1 Class Hierarchy                                                                       | 4        |
| <b>3 Class Index</b>                                                                      | <b>5</b> |
| 3.1 Class List                                                                            | 5        |
| <b>4 File Index</b>                                                                       | <b>7</b> |
| 4.1 File List                                                                             | 7        |
| <b>5 Class Documentation</b>                                                              | <b>9</b> |
| 5.1 processingmodules.AnimationGenerator.AnimationGenerator Class Reference               | 9        |
| 5.1.1 Detailed Description                                                                | 10       |
| 5.1.2 Member Function Documentation                                                       | 11       |
| 5.2 benchmark_options_t Struct Reference                                                  | 12       |
| 5.3 cmdline_args_t Struct Reference                                                       | 12       |
| 5.4 constant_options_t Struct Reference                                                   | 13       |
| 5.4.1 Detailed Description                                                                | 13       |
| 5.5 processingmodules.CrossSectionGenerator.CrossSectionGenerator Class Reference         | 13       |
| 5.5.1 Detailed Description                                                                | 14       |
| 5.5.2 Member Function Documentation                                                       | 15       |
| 5.6 datastream_t Struct Reference                                                         | 16       |
| 5.6.1 Detailed Description                                                                | 16       |
| 5.7 processingmodules.DensityAnimationGenerator.DensityAnimationGenerator Class Reference | 17       |
| 5.7.1 Detailed Description                                                                | 18       |
| 5.7.2 Member Function Documentation                                                       | 18       |
| 5.8 derivative_computation_opts_t Struct Reference                                        | 19       |
| 5.8.1 Detailed Description                                                                | 20       |
| 5.9 derivative_weight_t Struct Reference                                                  | 20       |

|                                                                                                |    |
|------------------------------------------------------------------------------------------------|----|
| 5.9.1 Detailed Description . . . . .                                                           | 20 |
| 5.10 distribution_options_t Struct Reference . . . . .                                         | 20 |
| 5.10.1 Detailed Description . . . . .                                                          | 21 |
| 5.11 double_3d_t Union Reference . . . . .                                                     | 21 |
| 5.11.1 Detailed Description . . . . .                                                          | 22 |
| 5.12 feature_positions_t Struct Reference . . . . .                                            | 22 |
| 5.12.1 Detailed Description . . . . .                                                          | 22 |
| 5.13 feature_velocity_t Struct Reference . . . . .                                             | 22 |
| 5.13.1 Detailed Description . . . . .                                                          | 23 |
| 5.14 finds_opts_t Struct Reference . . . . .                                                   | 23 |
| 5.14.1 Detailed Description . . . . .                                                          | 23 |
| 5.15 fish_system_t Struct Reference . . . . .                                                  | 23 |
| 5.15.1 Detailed Description . . . . .                                                          | 24 |
| 5.16 integration_opts_t Struct Reference . . . . .                                             | 24 |
| 5.16.1 Detailed Description . . . . .                                                          | 24 |
| 5.17 linear_octree_builder_t Struct Reference . . . . .                                        | 24 |
| 5.17.1 Detailed Description . . . . .                                                          | 25 |
| 5.18 linear_octree_t Struct Reference . . . . .                                                | 25 |
| 5.18.1 Detailed Description . . . . .                                                          | 26 |
| 5.19 mat3_t Struct Reference . . . . .                                                         | 26 |
| 5.19.1 Detailed Description . . . . .                                                          | 26 |
| 5.20 processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot Class Reference . . . . . | 26 |
| 5.20.1 Detailed Description . . . . .                                                          | 27 |
| 5.20.2 Member Function Documentation . . . . .                                                 | 27 |
| 5.21 morton_key_t Struct Reference . . . . .                                                   | 29 |
| 5.21.1 Detailed Description . . . . .                                                          | 29 |
| 5.22 named_enum_t Struct Reference . . . . .                                                   | 29 |
| 5.22.1 Detailed Description . . . . .                                                          | 29 |
| 5.23 orientation_options_t Struct Reference . . . . .                                          | 30 |
| 5.23.1 Detailed Description . . . . .                                                          | 30 |
| 5.24 processingmodules.ProcessingModule.ProcessingModule Class Reference . . . . .             | 30 |
| 5.24.1 Detailed Description . . . . .                                                          | 30 |
| 5.24.2 Member Function Documentation . . . . .                                                 | 31 |
| 5.25 processingmodules.SnapshotGenerator.SnapshotGenerator Class Reference . . . . .           | 32 |

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| 5.25.1 Detailed Description . . . . .                                          | 33        |
| 5.25.2 Member Function Documentation . . . . .                                 | 33        |
| 5.26 swimmer_features_t Struct Reference . . . . .                             | 34        |
| 5.26.1 Detailed Description . . . . .                                          | 35        |
| 5.27 swimmer_t Struct Reference . . . . .                                      | 35        |
| 5.27.1 Detailed Description . . . . .                                          | 35        |
| 5.28 system_derivative_t Struct Reference . . . . .                            | 35        |
| 5.28.1 Detailed Description . . . . .                                          | 36        |
| 5.29 system_trajectory_t Struct Reference . . . . .                            | 36        |
| 5.29.1 Detailed Description . . . . .                                          | 36        |
| 5.30 processingmodules.TrajectoryPlot.TrajectoryPlot Class Reference . . . . . | 36        |
| 5.30.1 Detailed Description . . . . .                                          | 37        |
| 5.30.2 Member Function Documentation . . . . .                                 | 37        |
| <b>6 File Documentation</b>                                                    | <b>39</b> |
| 6.1 include/finds/barneshut.h File Reference . . . . .                         | 39        |
| 6.1.1 Detailed Description . . . . .                                           | 40        |
| 6.1.2 Function Documentation . . . . .                                         | 40        |
| 6.2 include/finds/constants.h File Reference . . . . .                         | 42        |
| 6.2.1 Detailed Description . . . . .                                           | 42        |
| 6.3 include/finds/datastream.h File Reference . . . . .                        | 43        |
| 6.3.1 Detailed Description . . . . .                                           | 43        |
| 6.3.2 Function Documentation . . . . .                                         | 44        |
| 6.4 include/finds/derivative.h File Reference . . . . .                        | 47        |
| 6.4.1 Detailed Description . . . . .                                           | 48        |
| 6.4.2 Function Documentation . . . . .                                         | 48        |
| 6.5 include/finds/error.h File Reference . . . . .                             | 50        |
| 6.5.1 Detailed Description . . . . .                                           | 51        |
| 6.5.2 Macro Definition Documentation . . . . .                                 | 51        |
| 6.6 include/finds/evaluation.h File Reference . . . . .                        | 52        |
| 6.6.1 Detailed Description . . . . .                                           | 53        |
| 6.6.2 Function Documentation . . . . .                                         | 53        |
| 6.7 include/finds/finds.h File Reference . . . . .                             | 53        |
| 6.8 include/finds/integration.h File Reference . . . . .                       | 54        |
| 6.8.1 Detailed Description . . . . .                                           | 55        |

---

|                                                               |    |
|---------------------------------------------------------------|----|
| 6.8.2 Macro Definition Documentation . . . . .                | 55 |
| 6.8.3 Variable Documentation . . . . .                        | 56 |
| 6.9 include/finds/interaction.h File Reference . . . . .      | 56 |
| 6.9.1 Detailed Description . . . . .                          | 56 |
| 6.10 include/finds/simulation.h File Reference . . . . .      | 56 |
| 6.10.1 Detailed Description . . . . .                         | 57 |
| 6.10.2 Function Documentation . . . . .                       | 57 |
| 6.11 include/finds/system.h File Reference . . . . .          | 58 |
| 6.11.1 Detailed Description . . . . .                         | 60 |
| 6.11.2 Function Documentation . . . . .                       | 60 |
| 6.12 include/finds/util.h File Reference . . . . .            | 68 |
| 6.12.1 Detailed Description . . . . .                         | 68 |
| 6.12.2 Macro Definition Documentation . . . . .               | 69 |
| 6.13 include/finds/vector.h File Reference . . . . .          | 70 |
| 6.13.1 Detailed Description . . . . .                         | 70 |
| 6.13.2 Macro Definition Documentation . . . . .               | 71 |
| 6.14 programs/benchmark.c File Reference . . . . .            | 71 |
| 6.14.1 Detailed Description . . . . .                         | 72 |
| 6.15 programs/benchmark_derivative.c File Reference . . . . . | 72 |
| 6.15.1 Detailed Description . . . . .                         | 73 |
| 6.15.2 Variable Documentation . . . . .                       | 73 |
| 6.16 programs/linker_test.c File Reference . . . . .          | 74 |
| 6.16.1 Detailed Description . . . . .                         | 74 |
| 6.17 programs/simulate.c File Reference . . . . .             | 75 |
| 6.17.1 Detailed Description . . . . .                         | 76 |
| 6.17.2 Variable Documentation . . . . .                       | 76 |
| 6.18 programs/validation.c File Reference . . . . .           | 76 |
| 6.18.1 Detailed Description . . . . .                         | 77 |
| 6.18.2 Macro Definition Documentation . . . . .               | 78 |
| 6.19 src/barneshut.c File Reference . . . . .                 | 78 |
| 6.19.1 Detailed Description . . . . .                         | 79 |
| 6.19.2 Macro Definition Documentation . . . . .               | 79 |
| 6.19.3 Function Documentation . . . . .                       | 80 |
| 6.20 src/datastream.c File Reference . . . . .                | 82 |
| 6.20.1 Detailed Description . . . . .                         | 83 |

|                                                        |     |
|--------------------------------------------------------|-----|
| 6.20.2 Function Documentation . . . . .                | 83  |
| 6.21 src/evaluation.c File Reference . . . . .         | 86  |
| 6.21.1 Detailed Description . . . . .                  | 86  |
| 6.21.2 Function Documentation . . . . .                | 86  |
| 6.22 src/integration.c File Reference . . . . .        | 87  |
| 6.22.1 Detailed Description . . . . .                  | 88  |
| 6.22.2 Function Documentation . . . . .                | 88  |
| 6.22.3 Variable Documentation . . . . .                | 92  |
| 6.23 src/simulation.c File Reference . . . . .         | 92  |
| 6.23.1 Detailed Description . . . . .                  | 93  |
| 6.23.2 Function Documentation . . . . .                | 93  |
| 6.24 src/system.c File Reference . . . . .             | 94  |
| 6.24.1 Detailed Description . . . . .                  | 95  |
| 6.24.2 Function Documentation . . . . .                | 96  |
| 6.24.3 Variable Documentation . . . . .                | 103 |
| 6.25 tests/main.c File Reference . . . . .             | 104 |
| 6.25.1 Detailed Description . . . . .                  | 104 |
| 6.26 tests/test_datastream.c File Reference . . . . .  | 104 |
| 6.26.1 Detailed Description . . . . .                  | 105 |
| 6.26.2 Function Documentation . . . . .                | 105 |
| 6.27 tests/test_derivative.c File Reference . . . . .  | 105 |
| 6.27.1 Detailed Description . . . . .                  | 106 |
| 6.28 tests/test_integration.c File Reference . . . . . | 106 |
| 6.28.1 Detailed Description . . . . .                  | 106 |
| 6.29 tests/test_octree.c File Reference . . . . .      | 107 |
| 6.29.1 Detailed Description . . . . .                  | 107 |
| 6.30 tests/test_simulation.c File Reference . . . . .  | 107 |
| 6.30.1 Detailed Description . . . . .                  | 108 |
| 6.31 tests/test_system.c File Reference . . . . .      | 108 |
| 6.31.1 Detailed Description . . . . .                  | 108 |

|              |            |
|--------------|------------|
| <b>Index</b> | <b>109</b> |
|--------------|------------|

# 1 FINDS: The Fish Interaction and Dynamics Simulator

FINDS was written to solve the problem of the unscalability of fish-like particle simulations used to understand the mechanics of fish swimming in schools, birds flying in groups, drones/bugs flying in swarms, and so on.

This fundamentally follows the N-body problem, as each fish-like particle (which I have equivalently called "fish automata," "autofish," "fish- particles," and even just "particles" in various places) affects each other fish-like particle in the simulation. At a group size of  $N=3$ , an analytical solution is no longer possible, and numerical calculation is needed. As the group size  $N$  increases, the computational complexity in performing these numerical calculations grows as a function of  $N^2$ .

As such, the fundamental goal of 'Fish' was to simplify the calculations to reduce the effect of this bottleneck on Floryan's research through the implementation of approximation algorithms and speed tricks commonly used in Particle dynamics, starting with the Barnes-Hut approximation, which decreases the computational complexity to  $O(N \log N)$ .

Long term, the goal for the project is to try and reach computational parity with modern particle dynamics codes used in Astrophysics (and the like), such as [REBOUND](#), [GADGET-4](#), and [similar](#) applied to fish simulation. This is, of course, an ambitious goal for a one-man project, but we'll see how this fares.

## 1.1 SETUP

All that is required to install FInDS is Docker and GNU Make. For Windows users, I highly recommend using [WSL](#) and following the Linux instructions to keep your life simple, but this program can be run purely on windows regardless due to containerization.

### 1.1.1 Setup Example for Debian-Derived Systems

#### 1. Download prerequisites (docker and make)

```
sudo apt update
sudo apt install git make
sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-doc \
    podman-docker containerd runc | cut -f1)

# Add Docker's official GPG key:
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o \
    /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <EOF
Types: deb
URIs: https://download.docker.com/linux/debian
```

```
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin \
    docker-compose-plugin
```

## 1. Clone the Repository

```
git clone --recurse-submodules https://github.com/mufaro3/finds.git
```

## 1. cd into the repository and build the container and the documentation

```
cd finds
make build
make docs
```

1. FINDS is now ready for use. Take a look at `make help` for options on what you can do further from there.

### 1.1.2 Docker

#### Docker - Linux

All that is needed is the Docker CLI. Follow the instructions on the [Docker Website](#).

#### Docker - Windows and MacOS

You probably will need Docker Desktop. Follow the instructions on the [Docker Website](#).

### 1.1.3 GNU Make

#### GNU Make - Linux and MacOS

Your system likely comes with Make preinstalled, but if not, follow the instructions provided by the GNU foundation (ideally, download it via your package manager) at [GNU](#).



## 1.2 Acknowledgements

Thanks to Mohamed Niged Mabrouk, Professor Daniel Floryan, and Alvin Ng for their guidance on this project. In particular, Alvin Ng's `grav_sim` was used extensively as a reference work for developing FINDS due to its similarity in implementation.

## 2 Hierarchical Index

### 2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

|                                      |           |
|--------------------------------------|-----------|
| <b>benchmark_options_t</b>           | <b>12</b> |
| <b>cmdline_args_t</b>                | <b>12</b> |
| <b>constant_options_t</b>            | <b>13</b> |
| <b>datastream_t</b>                  | <b>16</b> |
| <b>derivative_computation_opts_t</b> | <b>19</b> |
| <b>derivative_weight_t</b>           | <b>20</b> |
| <b>distribution_options_t</b>        | <b>20</b> |
| <b>double_3d_t</b>                   | <b>21</b> |
| <b>feature_positions_t</b>           | <b>22</b> |
| <b>feature_velocity_t</b>            | <b>22</b> |
| <b>finds_opts_t</b>                  | <b>23</b> |
| <b>fish_system_t</b>                 | <b>23</b> |
| <b>integration_opts_t</b>            | <b>24</b> |
| <b>linear_octree_builder_t</b>       | <b>24</b> |
| <b>linear_octree_t</b>               | <b>25</b> |
| <b>mat3_t</b>                        | <b>26</b> |
| <b>morton_key_t</b>                  | <b>29</b> |
| <b>named_enum_t</b>                  | <b>29</b> |

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>orientation_options_t</b>                                                 | <b>30</b> |
| <b>swimmer_features_t</b>                                                    | <b>34</b> |
| <b>swimmer_t</b>                                                             | <b>35</b> |
| <b>system_derivative_t</b>                                                   | <b>35</b> |
| <b>system_trajectory_t</b>                                                   | <b>36</b> |
| ABC                                                                          |           |
| <b>processingmodules.ProcessingModule.ProcessingModule</b>                   | <b>30</b> |
| <b>processingmodules.AnimationGenerator.AnimationGenerator</b>               | <b>9</b>  |
| <b>processingmodules.CrossSectionGenerator.CrossSectionGenerator</b>         | <b>13</b> |
| <b>processingmodules.DensityAnimationGenerator.DensityAnimationGenerator</b> | <b>17</b> |
| <b>processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot</b>       | <b>26</b> |
| <b>processingmodules.SnapshotGenerator.SnapshotGenerator</b>                 | <b>32</b> |
| <b>processingmodules.TrajectoryPlot.TrajectoryPlot</b>                       | <b>36</b> |

## 3 Class Index

### 3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>processingmodules.AnimationGenerator.AnimationGenerator</b>               | <b>9</b>  |
| <b>benchmark_options_t</b>                                                   | <b>12</b> |
| <b>cmdline_args_t</b>                                                        | <b>12</b> |
| <b>constant_options_t</b>                                                    |           |
| Constant generation options struct                                           | <b>13</b> |
| <b>processingmodules.CrossSectionGenerator.CrossSectionGenerator</b>         | <b>13</b> |
| <b>datastream_t</b>                                                          |           |
| File I/O structure for HDF5 output                                           | <b>16</b> |
| <b>processingmodules.DensityAnimationGenerator.DensityAnimationGenerator</b> | <b>17</b> |

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| <b>derivative_computation_opts_t</b>                                     |    |
| Derivative computation options struct                                    | 19 |
| <b>derivative_weight_t</b>                                               |    |
| Weight structure for computing weighted averages of derivatives          | 20 |
| <b>distribution_options_t</b>                                            |    |
| Distribution generation options struct                                   | 20 |
| <b>double_3d_t</b>                                                       |    |
| 3-dimensional cartesian vector implementation                            | 21 |
| <b>feature_positions_t</b>                                               |    |
| Vector of feature positions                                              | 22 |
| <b>feature_velocity_t</b>                                                |    |
| Local alias for the derivative of <b>feature_positions_t</b> for clarity | 22 |
| <b>finds_opts_t</b>                                                      | 23 |
| <b>fish_system_t</b>                                                     |    |
| Struct for a system of swimmers (i.e., a vector of swimmers)             | 23 |
| <b>integration_opts_t</b>                                                |    |
| Struct for integration options                                           | 24 |
| <b>linear_octree_builder_t</b>                                           |    |
| A variable-sized wrapper for building linear octrees                     | 24 |
| <b>linear_octree_t</b>                                                   |    |
| Linear octree implementation for swimmer data                            | 25 |
| <b>mat3_t</b>                                                            |    |
| 3x3 matrix implementation                                                | 26 |
| <b>processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot</b>   | 26 |
| <b>morton_key_t</b>                                                      |    |
| This maps morton codes to node indices in a linear octree                | 29 |
| <b>named_enum_t</b>                                                      |    |
| Enum to name string mapping struct                                       | 29 |
| <b>orientation_options_t</b>                                             |    |
| Orientation generation options struct                                    | 30 |
| <b>processingmodules.ProcessingModule.ProcessingModule</b>               | 30 |
| <b>processingmodules.SnapshotGenerator.SnapshotGenerator</b>             | 32 |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>swimmer_features_t</b>                                    |           |
| Structure of feature (source and sink) positions             | <b>34</b> |
| <b>swimmer_t</b>                                             |           |
| Structure representing an individual swimmer                 | <b>35</b> |
| <b>system_derivative_t</b>                                   |           |
| System derivative (translational and rotational derivatives) | <b>35</b> |
| <b>system_trajectory_t</b>                                   |           |
| Swimmer trajectory object for simplifying the coplanar cases | <b>36</b> |
| <b>processingmodules.TrajectoryPlot.TrajectoryPlot</b>       | <b>36</b> |

## 4 File Index

### 4.1 File List

Here is a list of all documented files with brief descriptions:

|                                                                                                  |           |
|--------------------------------------------------------------------------------------------------|-----------|
| <b>include/finds/barneshut.h</b>                                                                 |           |
| External velocity contribution calculation through Barnes-Hut                                    | <b>39</b> |
| <b>include/finds/constants.h</b>                                                                 |           |
| Various constants for the FINDS system                                                           | <b>42</b> |
| <b>include/finds/datastream.h</b>                                                                |           |
| I/O routines                                                                                     | <b>43</b> |
| <b>include/finds/derivative.h</b>                                                                |           |
| Derivative calculation                                                                           | <b>47</b> |
| <b>include/finds/error.h</b>                                                                     |           |
| Basic error handling module                                                                      | <b>50</b> |
| <b>include/finds/evaluation.h</b>                                                                |           |
| Derivative timing                                                                                | <b>52</b> |
| <b>include/finds/finds.h</b>                                                                     | <b>53</b> |
| <b>include/finds/integration.h</b>                                                               |           |
| Implementation of various adaptive and non-adaptive Runge-Kutta integrators                      | <b>54</b> |
| <b>include/finds/interaction.h</b>                                                               |           |
| Compuatation function for calculating the Laplacian interaction between two fish or two features | <b>56</b> |

---

|                                                                     |     |
|---------------------------------------------------------------------|-----|
| include/finds/ <b>simulation.h</b>                                  |     |
| Core simulation function                                            | 56  |
| include/finds/ <b>system.h</b>                                      |     |
| Core particle system data structures and operations                 | 58  |
| include/finds/ <b>util.h</b>                                        |     |
| Utility functions                                                   | 68  |
| include/finds/ <b>vector.h</b>                                      |     |
| Basic vector and matrix computations                                | 70  |
| programs/ <b>benchmark.c</b>                                        |     |
| Produces a benchmarking-plot for the entirety of FINDS              | 71  |
| programs/ <b>benchmark_derivative.c</b>                             |     |
| Times the computation of a single derivative                        | 72  |
| programs/ <b>linker_test.c</b>                                      |     |
| Just a linker test that includes all of the libraries used in FINDS | 74  |
| programs/ <b>simulate.c</b>                                         |     |
| The core entrypoint for performing a simulation                     | 75  |
| programs/ <b>validation.c</b>                                       |     |
| Produces the validation plots for FINDS                             | 76  |
| src/ <b>barneshut.c</b>                                             |     |
| External velocity contribution calculation through Barnes-Hut       | 78  |
| src/ <b>datastream.c</b>                                            |     |
| I/O routines                                                        | 82  |
| src/ <b>evaluation.c</b>                                            |     |
| Performance evaluation                                              | 86  |
| src/ <b>integration.c</b>                                           |     |
| Integrator implementations                                          | 87  |
| src/ <b>simulation.c</b>                                            |     |
| Core simulation routine                                             | 92  |
| src/ <b>system.c</b>                                                |     |
| Core particle system data structures and operations                 | 94  |
| tests/ <b>main.c</b>                                                |     |
| Entry-point for running unit tests                                  | 104 |
| tests/ <b>test_datastream.c</b>                                     |     |
| Unit tests for the Datastream I/O module                            | 104 |

|                                                                  |            |
|------------------------------------------------------------------|------------|
| <b>tests/test_derivative.c</b>                                   |            |
| Unit tests for the derivative and interaction calculation module | <b>105</b> |
| <b>tests/test_integration.c</b>                                  |            |
| Unit tests for the numeric integration module                    | <b>106</b> |
| <b>tests/test_octree.c</b>                                       |            |
| Unit tests for the linear octree generation                      | <b>107</b> |
| <b>tests/test_simulation.c</b>                                   |            |
| Unit tests for the main simulation module                        | <b>107</b> |
| <b>tests/test_system.c</b>                                       |            |
| Unit tests for the fish system generation module                 | <b>108</b> |

## 5 Class Documentation

### 5.1 processingmodules.AnimationGenerator.AnimationGenerator Class Reference

#### Public Member Functions

- **\_\_init\_\_** (self, \*int fps=30, int dpi=300, str video\_output\_filename='animation3d.mp4', ArrayLike max\_bounds=[100, 100, 100], int particle\_radius=30, int orientation\_width=5, int orientation\_length=5, float padding=0.1, tuple[int, int] figsize=(8, 8), str particle\_color='tab:cyan', str orientation\_color='tab:orange')
- None **begin** (self, Path output\_dir, Optional[bool] use\_pdf)
- None **append\_state** (self, dict system, float time, int frame, int num\_frames)
- None **end** (self)

#### Public Attributes

- **fps**
- **dpi**
- **video\_output\_filename**
- **max\_bounds**
- **particle\_radius**
- **orientation\_width**
- **orientation\_length**
- **padding**
- **figsize**
- **particle\_color**

- **orientation\_color**
- **fig**
- **ax**
- **scatter\_plot**
- **quiver\_plot**
- **title\_text**
- **output\_filename**
- **writer**

#### Public Attributes inherited from

**processingmodules.ProcessingModule.ProcessingModule**

- **output\_dir**

### 5.1.1 Detailed Description

Generates a 3-D animation of the full fish simulation, taking (0,0) to be the origin. Allows for rapid customization of simulation colors, viewing, and debug parameters as well.

#### Attributes

=====

**fps:** int

The frames-per-second of the simulation, or how fast the animation will be. Default is 30 FPS.

**dpi:** int

The resolution of the video in dots per inch. Defaults to 300.

**video\_output\_filename:** str

The output filename of the video to generate. Default is :code:`animation3d.mp4`

**max\_bounds:** ArrayLike

The maximum (unsigned) bounds for the perspective. Putting in :math:`(x\_0, y\_0, z\_0)` means that the :math:`x`-axis varies from :math:`-x\_0` to :math:`x\_0`, and so on. Default is :math:`(100, 100, 100)`.

**particle\_radius:** int

The radius for each particle (default 30).

**orientation\_width:** int

The width for the line depicting the orientation (default 5).

**orientation\_length:** int

The length for the line depicting the orientation (default 5).

**padding:** float

The padding for the viewport/unit box (default 0.1).

`figsize: tuple[int, int]`  
The size of the resulting figure (not the viewport, default :code:`(8,8)`).

`particle_color: str`  
The color of the particle, default :code:`tab:cyan`.

`orientation_color: str`  
The color of the orientation line, default :code:`tab:orange`.

### 5.1.2 Member Function Documentation

#### **append\_state()**

```
None processingmodules.AnimationGenerator.AnimationGenerator.append_state (
    self,
    dict system,
    float time,
    int frame,
    int num_frames )
```

Draws the current state to the animation.

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

#### **begin()**

```
None processingmodules.AnimationGenerator.AnimationGenerator.begin (
    self,
    Path output_dir,
    Optional[bool] use_pdf )
```

Sets up the Matplotlib animation for rendering.

:type max\_bounds: ArrayLike

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).



**end()**

```
None processingmodules.AnimationGenerator.AnimationGenerator.end (
    self )
```

Closes the animation and saves the file.

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

The documentation for this class was generated from the following file:

- [scripts/processingmodules/AnimationGenerator.py](#)

## 5.2 benchmark\_options\_t Struct Reference

### Public Attributes

- `size_t` **size**
- [derivative\\_computation\\_opts\\_t](#) **dc\_opts**

The documentation for this struct was generated from the following file:

- [programs/benchmark\\_derivative.c](#)

## 5.3 cmdline\_args\_t Struct Reference

### Public Attributes

- `char *` **config\_filepath**

The documentation for this struct was generated from the following file:

- [programs/simulate.c](#)

## 5.4 constant\_options\_t Struct Reference

Constant generation options struct.

```
#include <system.h>
```

### Public Attributes

- bool **random\_length\_selection**
- float **min\_length**
- float **max\_length**
- float **uniform\_length**
- bool **random\_volumetric\_flow\_selection**
- float **min\_sigma**
- float **max\_sigma**
- float **uniform\_sigma**

### 5.4.1 Detailed Description

Constant generation options struct.

The documentation for this struct was generated from the following file:

- include/finds/**system.h**

## 5.5 processingmodules.CrossSectionGenerator.CrossSectionGenerator Class Reference

### Public Member Functions

- **\_\_init\_\_** (self, \*float z\_thickness=0.25, float particle\_radius=30, float orientation\_↔ width=0.25, float orientation\_length=1, tuple[int, int] figsize=(10, 10), str particle\_↔ color='tab:cyan', str orientation\_color='tab:orange', str output\_filename="cross\_section", int dpi=200)
- **begin** (self, Path output\_dir, Optional[bool] use\_pdf)
- **append\_state** (self, NDAarray system, float time, int frame, int num\_frames)
- **end** (self)

## Public Attributes

- **z\_thickness**
- **particle\_radius**
- **orientation\_width**
- **orientation\_length**
- **figsize**
- **particle\_color**
- **orientation\_color**
- **output\_filename**
- **dpi**
- **output\_dir**
- **generated**

## Public Attributes inherited from

**processingmodules.ProcessingModule.ProcessingModule**

- **output\_dir**

### 5.5.1 Detailed Description

Generates a cross-section for the first frame along the xy-plane at  $z=0$ .

Attributes

=====

**particle\_radius**: int

The radius for each particle (default 30).

**orientation\_width**: float

The width for the line depicting the orientation (default 0.25).

**orientation\_length**: float

The length for the line depicting the orientation (default 5).

**figsize**: tuple[int,int]

The size of the resulting figure (not the viewport, default :math:`(10,10)`).

**particle\_color**: str

The color of the particle, default :code:`tab:cyan`.

**orientation\_color**: str

The color of the orientation line, default :code:`tab:orange`.

**output\_filename**: str

The output filename of the image to generate. Default is :code:`cross\_section`

**generated**: bool

Whether or not we have already generated the cross-section figure.

**dpi**: int

The resolution of the fig in dots per inch. Defaults to 200.

### 5.5.2 Member Function Documentation

#### append\_state()

```
processingmodules.CrossSectionGenerator.CrossSectionGenerator.append_state
(  
    self,  
    NDAarray system,  
    float time,  
    int frame,  
    int num_frames )
```

Appends the state  $\mathbf{X}$  and time  $t$  to the current process being generated (whether by animating it, including it in a computation, etc.).

:param *system*: The state of the system at time  $t$ .  
:type *system*: NDAarray

:param *time*: The time,  $t$ .  
:type *time*: float

:param *frame*: The index for this frame  
:type *frame*: int

:param *num\_frames*: The total number of frames for this simulation  
:type *num\_frames*: int

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

#### begin()

```
processingmodules.CrossSectionGenerator.CrossSectionGenerator.begin (  
    self,  
    Path output_dir,  
    Optional[bool] use_pdf )
```

Instructs the processing module to initialize such that it can produce data.

:param *output\_dir*: The output directory to write to.  
:type *output\_dir*: Path

:param *use\_pdf*: Whether or not to output to SVG.  
:type *use\_pdf*: bool

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

**end()**

```
processingmodules.CrossSectionGenerator.CrossSectionGenerator.end (
    self )
```

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

The documentation for this class was generated from the following file:

- [scripts/processingmodules/CrossSectionGenerator.py](#)

## 5.6 datastream\_t Struct Reference

File I/O structure for HDF5 output.

```
#include <datastream.h>
```

### Public Attributes

- bool **open**
- hid\_t **file**
- hid\_t **time\_dataset**
- hid\_t **position\_dataset**
- hid\_t **orientation\_dataset**
- hid\_t **length\_dataset**
- hid\_t **sigma\_dataset**
- size\_t **n\_particles**
- size\_t **n\_frames**

### 5.6.1 Detailed Description

File I/O structure for HDF5 output.

The documentation for this struct was generated from the following file:

- [include/finds/datastream.h](#)

## 5.7 processingmodules.DensityAnimationGenerator.DensityAnimationGenerator Class Reference

### Public Member Functions

- `__init__` (self, \*int fps=30, int dpi=300, int n\_bins=30, float max\_radius=100.0, tuple[int, int] figsize=(6, 4), str video\_output\_filename='density\_animation.mp4')
- `None begin` (self, Path output\_dir, Optional[bool] use\_pdf)
- `None append_state` (self, NDArray system, float time, int frame, int num\_frames)
- `None end` (self)

### Public Attributes

- `fps`
- `dpi`
- `n_bins`
- `max_radius`
- `figsize`
- `video_output_filename`
- `output_dir`
- `fig`
- `ax`
- `bins`
- `bars`
- `title_text`
- `output_filename`
- `writer`

Public Attributes inherited from  
`processingmodules.ProcessingModule.ProcessingModule`

- `output_dir`

### 5.7.1 Detailed Description

Produces an animation of the radial mass distribution of fish from the center-of-mass for each time step,  $\Delta t$ .

Attributes

=====

`fps: int`

The frames-per-second of the animation. Default 30 FPS.

`dpi: int`

The dots-per-inch of the animation. Default 300.

`n_bins: int`

The number of bins to use in the histogram. Default 30.

`max_radius: float`

The maximum radius to display on the histogram. Default 100.

`figsize: tuple[int,int]`

The Matplotlib figure dimensions for the video. Default (6,4).

`video_output_filename: str`

The filename to output the video to.

### 5.7.2 Member Function Documentation

#### **append\_state()**

None processingmodules.DensityAnimationGenerator.DensityAnimationGenerator.  
append\_state (

```
    self,
    NDArray system,
    float time,
    int frame,
    int num_frames )
```

Appends the state  $\mathbf{X}$  and time  $t$  to the current process being generated (whether by animating it, including it in a computation, etc.).

:param system: The state of the system at time  $t$ .  
:type system: NDArray

:param time: The time,  $t$ .  
:type time: float

:param frame: The index for this frame  
:type frame: int

:param num\_frames: The total number of frames for this simulation  
:type num\_frames: int

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

**begin()**

```
None processingmodules.DensityAnimationGenerator.DensityAnimationGenerator.↔
begin (
    self,
    Path output_dir,
    Optional[bool] use_pdf )
```

Instructs the processing module to initialize such that it can produce data.

```
:param output_dir: The output directory to write to.
:type  output_dir: Path

:param use_pdf: Whether or not to output to SVG.
:type  use_pdf: bool
```

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

**end()**

```
None processingmodules.DensityAnimationGenerator.DensityAnimationGenerator.↔
end (
    self )
```

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

The documentation for this class was generated from the following file:

- scripts/processingmodules/DensityAnimationGenerator.py

**5.8 derivative\_computation\_opts\_t Struct Reference**

Derivative computation options struct.

```
#include <derivative.h>
```

**Public Attributes**

- [interaction\\_computation\\_methods\\_e](#) method
- double **approximation\_threshold**
- double **precision**
- double **regularization\_epsilon**
- bool **regularize**



### 5.8.1 Detailed Description

Derivative computation options struct.

The documentation for this struct was generated from the following file:

- include/finds/derivative.h

## 5.9 derivative\_weight\_t Struct Reference

Weight structure for computing weighted averages of derivatives.

```
#include <derivative.h>
```

### Public Attributes

- double **coeff**
- system\_derivative\_t \* **deriv**

### 5.9.1 Detailed Description

Weight structure for computing weighted averages of derivatives.

The documentation for this struct was generated from the following file:

- include/finds/derivative.h

## 5.10 distribution\_options\_t Struct Reference

Distribution generation options struct.

```
#include <system.h>
```

### Public Attributes

- `distribution_type_e` type
- union {  
    double **radius**  
    double **side\_length**  
    double **size\_random**  
};
- union {  
    double **spacing**  
    double **abs\_bound**  
};

#### 5.10.1 Detailed Description

Distribution generation options struct.

The documentation for this struct was generated from the following file:

- `include/finds/system.h`

## 5.11 double\_3d\_t Union Reference

3-dimensional cartesian vector implementation

```
#include <vector.h>
```

### Public Attributes

- struct {  
    double **x**  
    double **y**  
    double **z**  
};
- double **data** [3]

### 5.11.1 Detailed Description

3-dimensional cartesian vector implementation

The documentation for this union was generated from the following file:

- include/finds/**vector.h**

## 5.12 feature\_positions\_t Struct Reference

Vector of feature positions.

```
#include <system.h>
```

### Public Attributes

- **swimmer\_features\_t** \* **swimmers**
- size\_t **size**

### 5.12.1 Detailed Description

Vector of feature positions.

The documentation for this struct was generated from the following file:

- include/finds/**system.h**

## 5.13 feature\_velocity\_t Struct Reference

Local alias for the derivative of **feature\_positions\_t** for clarity.

### Public Attributes

- **double\_3d\_t** \* **source**
- **double\_3d\_t** \* **sink**
- size\_t **size**

### 5.13.1 Detailed Description

Local alias for the derivative of `feature_positions_t` for clarity.

The documentation for this struct was generated from the following file:

- `src/derivative.c`

## 5.14 finds\_opts\_t Struct Reference

### Public Attributes

- `distribution_options_t` **dist\_opts**
- `orientation_options_t` **ori\_opts**
- `constant_options_t` **const\_opts**
- `derivative_computation_opts_t` **dc\_opts**
- `integration_opts_t` **int\_opts**
- `bool` **print\_output\_file**
- `bool` **print\_generated\_system**

### 5.14.1 Detailed Description

Simulation options for use in loading the TOML file.

The documentation for this struct was generated from the following file:

- `programs/simulate.c`

## 5.15 fish\_system\_t Struct Reference

Struct for a system of swimmers (i.e., a vector of swimmers).

```
#include <system.h>
```

### Public Attributes

- `swimmer_t` \* **swimmers**
- `size_t` **size**

### 5.15.1 Detailed Description

Struct for a system of swimmers (i.e., a vector of swimmers).

The documentation for this struct was generated from the following file:

- `include/finds/system.h`

## 5.16 `integration_opts_t` Struct Reference

Struct for integration options.

```
#include <integration.h>
```

### Public Attributes

- `integration_method_e` method
- double `eval_time_step`
- double `end_time`
- double `relative_error_tolerance`
- double `absolute_error_tolerance`
- bool `print_time_progression`

### 5.16.1 Detailed Description

Struct for integration options.

The documentation for this struct was generated from the following file:

- `include/finds/integration.h`

## 5.17 `linear_octree_builder_t` Struct Reference

A variable-sized wrapper for building linear octrees.

## Public Attributes

- size\_t **capacity**
- linear\_octree\_t **tree**

### 5.17.1 Detailed Description

A variable-sized wrapper for building linear octrees.

Growable builder that is trimmed to exact size once construction finishes. Defined to be variable-size so that the actual linear octree can be nicely fixed-size upon construction. Otherwise, it is completely identical to the linear octree.

The documentation for this struct was generated from the following file:

- src/barneshut.c

## 5.18 linear\_octree\_t Struct Reference

Linear octree implementation for swimmer data.

```
#include <barneshut.h>
```

## Public Attributes

- uint64\_t **count**
- double\_3d\_t \* **centers**
- double \* **side\_lengths**
- child\_indices\_t \* **child\_indices**
- bool \* **is\_leaf**
- size\_t \* **swimmer\_index**
- size\_t \* **num\_particles**
- double\_3d\_t \* **weighted\_positions\_sums**
- double\_3d\_t \* **weighted\_orientations\_sums**
- double \* **weighted\_length\_sums**
- double \* **volumetric\_flow\_rate\_sums**
- double\_3d\_t \* **source\_position\_avg**
- double\_3d\_t \* **sink\_position\_avg**

### 5.18.1 Detailed Description

Linear octree implementation for swimmer data.

The documentation for this struct was generated from the following file:

- include/finds/**barneshut.h**

## 5.19 mat3\_t Struct Reference

3x3 matrix implementation

```
#include <vector.h>
```

### Public Attributes

- double **data** [3][3]

### 5.19.1 Detailed Description

3x3 matrix implementation

The documentation for this struct was generated from the following file:

- include/finds/**vector.h**

## 5.20 processingmodules.MeanRadialDistancePlot.MeanRadialDistance↔ Plot Class Reference

### Public Member Functions

- **\_\_init\_\_** (self, \*int dpi=300, tuple[int, int] figsize=(10, 10), str output\_filename='mean\_↔radial\_distance')
- None **begin** (self, Path output\_dir, Optional[bool] use\_pdf)
- None **append\_state** (self, NDArray system, float time, int frame, int num\_frames)
- None **end** (self)

## Public Attributes

- **dpi**
- **figsize**
- **output\_filename**
- **output\_dir**
- **output\_filepath**
- **std\_radial\_distances**

## Public Attributes inherited from

**processingmodules.ProcessingModule.ProcessingModule**

- **output\_dir**

### 5.20.1 Detailed Description

Produces a plot of the mean radial mass distribution of fish from the center-of-mass as a function of time, including errorbars for the standard deviations.

#### Attributes

=====

dpi: int

The dots-per-inch of the animation. Default 300.

figsize: tuple[int,int]

The Matplotlib figure dimensions for the video. Default (10,10).

output\_filename: str

The filename to output the figure to.

### 5.20.2 Member Function Documentation

#### append\_state()

```
None processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot.append↵
_state (
    self,
    NDArray system,
    float time,
    int frame,
    int num_frames )
```



Appends the state  $\mathbf{X}$  and time  $t$  to the current process being generated (whether by animating it, including it in a computation, etc.).

:param system: The state of the system at time  $t$ .  
:type system: NDArray

:param time: The time,  $t$ .  
:type time: float

:param frame: The index for this frame  
:type frame: int

:param num\_frames: The total number of frames for this simulation  
:type num\_frames: int

Reimplemented from `processingmodules.ProcessingModule.ProcessingModule`.

### **begin()**

```
None processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot.begin
(  
    self,  
    Path output_dir,  
    Optional[bool] use_pdf )
```

Instructs the processing module to initialize such that it can produce data.

:param output\_dir: The output directory to write to.  
:type output\_dir: Path

:param use\_pdf: Whether or not to output to SVG.  
:type use\_pdf: bool

Reimplemented from `processingmodules.ProcessingModule.ProcessingModule`.

### **end()**

```
None processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot.end (  
    self )
```

Reimplemented from `processingmodules.ProcessingModule.ProcessingModule`.

The documentation for this class was generated from the following file:

- `scripts/processingmodules/MeanRadialDistancePlot.py`

## 5.21 morton\_key\_t Struct Reference

This maps morton codes to node indices in a linear octree.

### Public Attributes

- uint64\_t **morton\_code**
- size\_t **assoc\_node\_index**

### 5.21.1 Detailed Description

This maps morton codes to node indices in a linear octree.

The documentation for this struct was generated from the following file:

- src/[barneshut.c](#)

## 5.22 named\_enum\_t Struct Reference

Enum to name string mapping struct.

```
#include <util.h>
```

### Public Attributes

- int **enum\_value**
- char \* **name**

### 5.22.1 Detailed Description

Enum to name string mapping struct.

The documentation for this struct was generated from the following file:

- include/finds/[util.h](#)

## 5.23 orientation\_options\_t Struct Reference

Orientation generation options struct.

```
#include <system.h>
```

### Public Attributes

- `orientation_type_e` type
- float `angular_perturbation`

### 5.23.1 Detailed Description

Orientation generation options struct.

The documentation for this struct was generated from the following file:

- `include/finds/system.h`

## 5.24 processingmodules.ProcessingModule.ProcessingModule Class Reference

### Public Member Functions

- `begin` (self, Path output\_dir, Optional[bool] use\_pdf)
- None `append_state` (self, NDArray system, float time, int frame, int num\_frames)
- None `end` (self)

### Public Attributes

- `output_dir`

### 5.24.1 Detailed Description

This is an abstract class for all "processing modules" so that they can operate in parallel to reduce the amount of data read to the disk at one time (such as generating figures, performing calculations, etc).

### 5.24.2 Member Function Documentation

#### append\_state()

```
None processingmodules.ProcessingModule.ProcessingModule.append_state (
    self,
    NDAarray system,
    float time,
    int frame,
    int num_frames )
```

Appends the state  $\mathbf{X}$  and time  $t$  to the current process being generated (whether by animating it, including it in a computation, etc.).

:param system: The state of the system at time  $t$ .  
:type system: NDAarray

:param time: The time,  $t$ .  
:type time: float

:param frame: The index for this frame  
:type frame: int

:param num\_frames: The total number of frames for this simulation  
:type num\_frames: int

Reimplemented in [processingmodules.AnimationGenerator.AnimationGenerator](#), [processingmodules.CrossSectionGenerator.CrossSectionGenerator](#), [processingmodules.DensityAnimationGenerator.DensityAnimationGenerator](#), [processingmodules.MeanRadialDistributionGenerator.MeanRadialDistributionGenerator](#), [processingmodules.TrajectoryPlot.TrajectoryPlot](#), and [processingmodules.SnapshotGenerator.SnapshotGenerator](#).

#### begin()

```
processingmodules.ProcessingModule.ProcessingModule.begin (
    self,
    Path output_dir,
    Optional[bool] use_pdf )
```

Instructs the processing module to initialize such that it can produce data.

:param output\_dir: The output directory to write to.  
:type output\_dir: Path

:param use\_pdf: Whether or not to output to SVG.  
:type use\_pdf: bool

Reimplemented in [processingmodules.AnimationGenerator.AnimationGenerator](#), [processingmodules.CrossSectionGenerator.CrossSectionGenerator](#), [processingmodules.DensityAnimationGenerator.DensityAnimationGenerator](#), [processingmodules.MeanRadialDistributionGenerator.MeanRadialDistributionGenerator](#), [processingmodules.SnapshotGenerator.SnapshotGenerator](#), and [processingmodules.TrajectoryPlot.TrajectoryPlot](#).

**end()**

```
None processingmodules.ProcessingModule.ProcessingModule.end (
    self )
```

Reimplemented in [processingmodules.AnimationGenerator.AnimationGenerator](#).

The documentation for this class was generated from the following file:

- [scripts/processingmodules/ProcessingModule.py](#)

## 5.25 processingmodules.SnapshotGenerator.SnapshotGenerator Class Reference

### Public Member Functions

- **\_\_init\_\_** (self, \*str output\_filename="snapshot", str output\_filename\_ending="png", ArrayLike max\_bounds=[100, 100, 100], int particle\_radius=30, float orientation\_length=5, float padding=0.1, tuple[int, int] figsize=(8, 8), str particle\_color="tab:cyan", str orientation\_color="tab:orange")
- None **begin** (self, Path output\_dir, Optional[bool] use\_pdf)
- None **append\_state** (self, system, float time, int frame, int num\_frames)
- None **end** (self)

### Public Attributes

- **output\_filename**
- **output\_filename\_ending**
- **max\_bounds**
- **particle\_radius**
- **orientation\_length**
- **padding**
- **figsize**
- **particle\_color**
- **orientation\_color**
- **output\_dir**
- **first**
- **middle**
- **last**

### Public Attributes inherited from processingmodules.ProcessingModule.ProcessingModule

- `output_dir`

### Protected Member Functions

- `_draw_snapshot` (self, positions, orientations, time, index)

#### 5.25.1 Detailed Description

Generates a three-panel snapshot of the simulation showing the first, middle, and last simulation frames.

#### 5.25.2 Member Function Documentation

##### `append_state()`

```
None processingmodules.SnapshotGenerator.SnapshotGenerator.append_state (
    self,
    system,
    float time,
    int frame,
    int num_frames )
```

Appends the state  $\mathbf{X}$  and time  $t$  to the current process being generated (whether by animating it, including it in a computation, etc.).

:param system: The state of the system at time  $t$ .  
:type system: NDArray

:param time: The time,  $t$ .  
:type time: float

:param frame: The index for this frame  
:type frame: int

:param num\_frames: The total number of frames for this simulation  
:type num\_frames: int

Reimplemented from processingmodules.ProcessingModule.ProcessingModule.

**begin()**

```
None processingmodules.SnapshotGenerator.SnapshotGenerator.begin (
    self,
    Path output_dir,
    Optional[bool] use_pdf )
```

Instructs the processing module to initialize such that it can produce data.

```
:param output_dir: The output directory to write to.
:type  output_dir: Path

:param use_pdf: Whether or not to output to SVG.
:type  use_pdf: bool
```

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

**end()**

```
None processingmodules.SnapshotGenerator.SnapshotGenerator.end (
    self )
```

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

The documentation for this class was generated from the following file:

- `scripts/processingmodules/SnapshotGenerator.py`

## 5.26 swimmer\_features\_t Struct Reference

Structure of feature (source and sink) positions.

```
#include <system.h>
```

**Public Attributes**

- [double\\_3d\\_t](#) **source**
- [double\\_3d\\_t](#) **sink**

### 5.26.1 Detailed Description

Structure of feature (source and sink) positions.

The documentation for this struct was generated from the following file:

- include/finds/system.h

## 5.27 swimmer\_t Struct Reference

Structure representing an individual swimmer.

```
#include <system.h>
```

### Public Attributes

- double\_3d\_t position
- double\_3d\_t orientation
- double volumetric\_flow\_rate
- double length
- double sp\_speed

### 5.27.1 Detailed Description

Structure representing an individual swimmer.

The documentation for this struct was generated from the following file:

- include/finds/system.h

## 5.28 system\_derivative\_t Struct Reference

System derivative (translational and rotational derivatives).

```
#include <derivative.h>
```



### Public Attributes

- `double_3d_t` \* **translational**
- `double_3d_t` \* **rotational**
- `size_t` **size**

#### 5.28.1 Detailed Description

System derivative (translational and rotational derivatives).

The documentation for this struct was generated from the following file:

- `include/finds/derivative.h`

## 5.29 `system_trajectory_t` Struct Reference

Swimmer trajectory object for simplifying the coplanar cases.

### Public Attributes

- `size_t` **num\_frames**
- `size_t` **num\_swimmers**
- `swimmer_t` \*\* **data**

#### 5.29.1 Detailed Description

Swimmer trajectory object for simplifying the coplanar cases.

The documentation for this struct was generated from the following file:

- `programs/validation.c`

## 5.30 `processingmodules.TrajectoryPlot.TrajectoryPlot` Class Reference

### Public Member Functions

- `__init__` (self, \*int dpi=300, tuple[int, int] figsize=(10, 10), str output\_filename='trajectory↔\_plot', int particle\_radius=30)
- None `begin` (self, Path output\_dir, Optional[bool] use\_pdf)
- None `append_state` (self, NDArray system, float time, int frame, int num\_frames)
- None `end` (self)

## Public Attributes

- **dpi**
- **figsize**
- **output\_filename**
- **particle\_radius**
- **output\_dir**
- **output\_filepath**
- **position\_frames**

## Public Attributes inherited from

**processingmodules.ProcessingModule.ProcessingModule**

- **output\_dir**

### 5.30.1 Detailed Description

Produces a plot of the mean radial mass distribution of fish from the center-of-mass as a function of time, including errorbars for the standard deviations.

#### Attributes

=====

**dpi:** int

The dots-per-inch of the animation. Default 300.

**figsize:** tuple[int,int]

The Matplotlib figure dimensions for the video. Default (10,10).

**plot\_output\_filename:** str

The filename to output the figure to (default :code: 'trajectory\_plot.png').

**particle\_radius:** int

The radius of each fish-particle (default 30).

### 5.30.2 Member Function Documentation

#### **append\_state()**

```
None processingmodules.TrajectoryPlot.TrajectoryPlot.append_state (
    self,
    NDArray system,
    float time,
    int frame,
    int num_frames )
```

Appends the state  $\mathbf{X}$  and time  $t$  to the current process being generated (whether by animating it, including it in a computation, etc.).

:param system: The state of the system at time  $t$ .  
:type system: NDArray

:param time: The time,  $t$ .  
:type time: float

:param frame: The index for this frame  
:type frame: int

:param num\_frames: The total number of frames for this simulation  
:type num\_frames: int

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

### **begin()**

```
None processingmodules.TrajectoryPlot.TrajectoryPlot.begin (
    self,
    Path output_dir,
    Optional[bool] use_pdf )
```

Instructs the processing module to initialize such that it can produce data.

:param output\_dir: The output directory to write to.  
:type output\_dir: Path

:param use\_pdf: Whether or not to output to SVG.  
:type use\_pdf: bool

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

### **end()**

```
None processingmodules.TrajectoryPlot.TrajectoryPlot.end (
    self )
```

Reimplemented from [processingmodules.ProcessingModule.ProcessingModule](#).

The documentation for this class was generated from the following file:

- `scripts/processingmodules/TrajectoryPlot.py`

## 6 File Documentation

### 6.1 include/finds/barneshut.h File Reference

External velocity contribution calculation through Barnes-Hut.

```
#include <stdint.h>
#include "system.h"
#include "vector.h"
#include "error.h"
```

#### Classes

- struct **linear\_octree\_t**  
*Linear octree implementation for swimmer data.*

#### Macros

- #define **OCTREE\_NO\_CHILD** UINT64\_MAX  
*Sentinel value for "no child in this octant".*

#### Typedefs

- typedef uint64\_t **child\_indices\_t**[8]  
*Typedef for size-8 indices.*

#### Functions

- bool **linear\_octree\_check\_nan** (const **linear\_octree\_t** \*restrict tree, bool verbose)  
*Checks if a linear octree has any NaN values.*
- void **linear\_octree\_print** (const **linear\_octree\_t** \*restrict tree)  
*Prints a full octree to STDOUT (for debugging).*
- **linear\_octree\_t** \* **linear\_octree\_build** (const **fish\_system\_t** \*restrict system)  
*Builds a linear octree from a fish system.*
- void **linear\_octree\_destroy** (**linear\_octree\_t** \*\*octree\_ptr)  
*De-allocates a linear octree.*
- void **linear\_octree\_compute\_vel\_contrib** (const **linear\_octree\_t** \*restrict tree, const size\_t swimmer\_i\_index, const **double\_3d\_t** swimmer\_i\_pos, const **swimmer\_features\_t** features\_i, const double approx\_ratio, const bool regularize, const double epsilon, **double\_3d\_t** \*restrict external\_source, **double\_3d\_t** \*restrict external\_sink)  
*Computes the velocity contribution on a swimmer by all of the non-self swimmers in the system.*

### 6.1.1 Detailed Description

External velocity contribution calculation through Barnes-Hut.

This file contains an implementation of the Barnes-Hut algorithm for external velocity contribution using linear octrees as a backend.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.1.2 Function Documentation

#### **linear\_octree\_build()**

```
linear_octree_t * linear_octree_build (
    const fish_system_t *restrict system )
```

Builds a linear octree from a fish system.

#### Parameters

|    |               |                                    |
|----|---------------|------------------------------------|
| in | <i>system</i> | The fish system to construct from. |
|----|---------------|------------------------------------|

#### Returns

The constructed linear octree.

#### **linear\_octree\_check\_nan()**

```
bool linear_octree_check_nan (
    const linear_octree_t *restrict tree,
    bool verbose )
```

Checks if a linear octree has any NaN values.

**Parameters**

|    |                |                                                      |
|----|----------------|------------------------------------------------------|
| in | <i>tree</i>    | The octree.                                          |
| in | <i>verbose</i> | Whether to print the exact value which contains NaN. |

**Returns**

Whether or not any NaN values were found inside the tree.

**linear\_octree\_compute\_vel\_contrib()**

```
void linear_octree_compute_vel_contrib (
    const linear_octree_t *restrict tree,
    const size_t swimmer_i_index,
    const double_3d_t swimmer_i_pos,
    const swimmer_features_t features_i,
    const double approx_ratio,
    const bool regularize,
    const double eps,
    double_3d_t *restrict external_source,
    double_3d_t *restrict external_sink )
```

Computes the velocity contribution on a swimmer by all of the non-self swimmers in the system.

**Parameters**

|     |                        |                                                |
|-----|------------------------|------------------------------------------------|
| in  | <i>tree</i>            | The linear octree.                             |
| in  | <i>swimmer_i_index</i> | The swimmer index for swimmer i.               |
| in  | <i>swimmer_i_pos</i>   | The position of swimmer i.                     |
| in  | <i>features_i</i>      | The feature positions of swimmer i.            |
| in  | <i>approx_ratio</i>    | The Barnes-Hut approximation ratio.            |
| in  | <i>regularize</i>      | Whether or not to use regularized interaction. |
| in  | <i>eps</i>             | Regularization Epsilon                         |
| out | <i>external_source</i> | The output vec. for external source contrib.   |
| out | <i>external_sink</i>   | The output vec. for external sink contrib.     |

**linear\_octree\_destroy()**

```
void linear_octree_destroy (
    linear_octree_t ** octree_ptr )
```

De-allocates a linear octree.

**Parameters**

|                  |                         |                                     |
|------------------|-------------------------|-------------------------------------|
| <code>out</code> | <code>octree_ptr</code> | The point to the octree to destroy. |
|------------------|-------------------------|-------------------------------------|

**linear\_octree\_print()**

```
void linear_octree_print (
    const linear_octree_t *restrict tree )
```

Prints a full octree to STDOUT (for debugging).

**Parameters**

|                 |                   |                      |
|-----------------|-------------------|----------------------|
| <code>in</code> | <code>tree</code> | The octree to print. |
|-----------------|-------------------|----------------------|

## 6.2 include/finds/constants.h File Reference

Various constants for the FINDS system.

**Macros**

- `#define ROOT_OUTPUT_PATH "output"`
- `#define SIMULATIONS_OUTPUT_PATH "simulations"`
- `#define DATAFILE_NAME "data.h5"`
- `#define SAFETY 0.9f`
- `#define MODE_RW_USERONLY 0755`

### 6.2.1 Detailed Description

Various constants for the FINDS system.

**Author**

Mufaro Machaya

License: MIT

## 6.3 include/finds/datastream.h File Reference

I/O routines.

```
#include <hdf5.h>
#include <stdbool.h>
#include "system.h"
#include "error.h"
```

### Classes

- struct **datastream\_t**  
*File I/O structure for HDF5 output.*

### Functions

- **error\_e mkdir\_p** (const char \*path, const mode\_t mode)  
*C implementation of "mkdir -p" command.*
- **error\_e datastream\_open\_file** (datastream\_t \*stream, const char \*filename)  
*Opens a dataset file for reading.*
- **error\_e datastream\_read\_frame** (datastream\_t \*stream, const size\_t frame, fish\_system\_t \*\*dest\_ptr, double \*time)
- **error\_e datastream\_create\_file** (datastream\_t \*stream, const char \*filename, const size\_t N)  
*Creates the root file for the datastream and marks it as open.*
- **error\_e datastream\_close** (datastream\_t \*stream)  
*Closes this datastream.*
- **error\_e datastream\_write\_system** (datastream\_t \*stream, const fish\_system\_t \*system, const double time)  
*Writes the state of a single system out to the datastream.*

### 6.3.1 Detailed Description

I/O routines.

This contains the implementation of the datastream object, which is used for creating and writing to simulation data files.

#### Author

Mufaro Machaya

License: MIT



### 6.3.2 Function Documentation

#### **datastream\_close()**

```
error_e datastream_close (
    datastream_t * stream )
```

Closes this datastream.

##### Parameters

|     |               |                      |
|-----|---------------|----------------------|
| out | <i>stream</i> | The stream to close. |
|-----|---------------|----------------------|

##### Returns

The error code for this process.

#### **datastream\_create\_file()**

```
error_e datastream_create_file (
    datastream_t * stream,
    const char * filename,
    const size_t N )
```

Creates the root file for the datastream and marks it as open.

##### Parameters

|     |                 |                                                  |
|-----|-----------------|--------------------------------------------------|
| out | <i>stream</i>   | The datastream to open.                          |
| in  | <i>filename</i> | The output filename for the dataset.             |
| in  | <i>N</i>        | The number of swimmers to track in this dataset. |

##### Returns

The error code for this process.

#### **datastream\_open\_file()**

```
error_e datastream_open_file (
    datastream_t * stream,
    const char * filename )
```

Opens a dataset file for reading.

**Parameters**

|     |                 |                            |
|-----|-----------------|----------------------------|
| out | <i>stream</i>   | The datastream to read to. |
| in  | <i>filename</i> | The file to open.          |

**Returns**

The error code.

**datastream\_write\_system()**

```
error_e datastream_write_system (
    datastream_t * stream,
    const fish_system_t * system,
    const double time )
```

Writes the state of a single system out to the datastream.

**Parameters**

|     |               |                             |
|-----|---------------|-----------------------------|
| out | <i>stream</i> | The datastream to write to. |
| in  | <i>system</i> | The system state to write.  |
| in  | <i>time</i>   | The time marker to write.   |

**Returns**

The error code for this operation.

**mkdir\_p()**

```
error_e mkdir_p (
    const char * path,
    const mode_t mode )
```

C implementation of "mkdir -p" command.

Note: I'll be honest, I just stole this from generative AI. It works quite okay, though. Another possible implementation could just directly use `mkdir -p` via a system call (as this code doesn't need to be portable due to running in a virtual machine).

### Parameters

|    |             |                            |
|----|-------------|----------------------------|
| in | <i>path</i> | The folder path to create. |
| in | <i>mode</i> | The mkdir creation mode.   |

### Returns

The error code for this process.

## 6.4 include/finds/derivative.h File Reference

Derivative calculation.

```
#include <string.h>
#include <stdlib.h>
#include "vector.h"
#include "system.h"
#include "util.h"
```

### Classes

- struct `system_derivative_t`  
*System derivative (translational and rotational derivatives).*
- struct `derivative_computation_opts_t`  
*Derivative computation options struct.*
- struct `derivative_weight_t`  
*Weight structure for computing weighted averages of derivatives.*

### Enumerations

- enum `interaction_computation_methods_e` { `BRUTE_FORCE` , `BARNES_HUT` , `FAST↔_MULTIPOLE_METHOD` , `INTER_COMP_METHODS_COUNT` }  
*Interaction computation methods list.*

## Functions

- `system_derivative_t * compute_system_derivative` (const `fish_system_t` \*system, const `derivative_computation_opts_t` dc\_opts)  
*Computes the derivative for a fish system.*
- bool `derivative_has_nan` (const `system_derivative_t` \*derivative)  
*Determines whether a system derivative contains a NaN or Inf value (for debugging).*
- void `derivative_destroy` (`system_derivative_t` \*\*derivative)  
*De-allocates a derivative object.*
- void `derivative_print` (const `system_derivative_t` \*derivative)  
*Prints a derivative (for debugging).*
- `system_derivative_t * derivative_average` (const `derivative_weight_t` terms[], const size\_t N, const size\_t len)  
*Computes the weighted average of a derivative.*

## Variables

- const `named_enum_t` `INTER_COMP_METHODS_TABLE` [INTER\_COMP\_METHODS\_COUNT]  
*Lookup table for the interaction computation methods.*

### 6.4.1 Detailed Description

Derivative calculation.

This file contains the core routine for computing the derivative of the fish system.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.4.2 Function Documentation

#### `compute_system_derivative()`

```
system_derivative_t * compute_system_derivative (
    const fish_system_t * system,
    const derivative_computation_opts_t dc_opts )
```

Computes the derivative for a fish system.

**Parameters**

|    |                |                                          |
|----|----------------|------------------------------------------|
| in | <i>system</i>  | The system to compute the derivative of. |
| in | <i>dc_opts</i> | The derivative computation options.      |

**derivative\_average()**

```
system_derivative_t * derivative_average (
    const derivative_weight_t terms[],
    const size_t N,
    const size_t len )
```

Computes the weighted average of a derivative.

**Parameters**

|    |              |                                    |
|----|--------------|------------------------------------|
| in | <i>terms</i> | The terms to compute on.           |
| in | <i>N</i>     | The size of the outupt derivative. |

**Returns**

The weighted average derivative.

**derivative\_destroy()**

```
void derivative_destroy (
    system_derivative_t ** derivative_ptr )
```

De-allocates a derivative object.

**Parameters**

|     |                       |                                       |
|-----|-----------------------|---------------------------------------|
| out | <i>derivative_ptr</i> | The pointer to the derivative object. |
|-----|-----------------------|---------------------------------------|

**derivative\_has\_nan()**

```
bool derivative_has_nan (
    const system_derivative_t * derivative )
```

Determines whether a system derivative contains a NaN or Inf value (for debugging).

**Parameters**

|    |                   |                        |
|----|-------------------|------------------------|
| in | <i>derivative</i> | The system derivative. |
|----|-------------------|------------------------|

**Returns**

Whether the system derivative contains a non-finite value.

**derivative\_print()**

```
void derivative_print (
    const system_derivative_t * derivative )
```

Prints a derivative (for debugging).

**Parameters**

|    |                   |                        |
|----|-------------------|------------------------|
| in | <i>derivative</i> | The system derivative. |
|----|-------------------|------------------------|

## 6.5 include/finds/error.h File Reference

Basic error handling module.

```
#include <errno.h>
```

**Macros**

- #define *WRAP\_HDF5\_CHECK*(errcode, jmp to, result)
- #define *WRAP\_CHECK*(errcode, jmp to, prev\_errcode)
- #define *RAISE\_ERROR*(errtype, errmsg)
- #define *RAISE\_ERROR\_ERRNO*(errtype, errmsg)
- #define *GOTO\_IF\_NULL*(errcode, jmp to, ptr)
- #define *WARN*(msg) fprintf(stderr, "WARNING: %s\n", msg)

## Enumerations

- enum `error_e` {  
`ERR_DATASTREAM_OPEN` , `ERR_DATASTREAM_CLOSED` , `ERR_DATASTREAM_WRITE` , `ERR_INVALID_ARG` ,  
`ERR_ALLOC` , `ERR_STATE_CHANGE` , `ERR_INVALID_STATE` , `ERR_FAILURE` ,  
`ERR_OK` , `ERR_TYPE_COUNT` }

*Enum for error types.*

### 6.5.1 Detailed Description

Basic error handling module.

This file contains some simple macros for producing and handling errors.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.5.2 Macro Definition Documentation

#### GOTO\_IF\_NULL

```
#define GOTO_IF_NULL(  
    errcode,  
    jmpto,  
    ptr )
```

#### Value:

```
{ if (ptr == NULL) {  
    errcode = ERR_FAILURE;  
    goto jmpto;  
}}
```

#### RAISE\_ERROR

```
#define RAISE_ERROR(  
    errtype,  
    errmsg )
```

#### Value:

```
{  
    fprintf(stderr, "%s:%d %s: %s\n",  
        __FILE__, __LINE__, __func__, (errmsg));  
    (errtype);  
}
```



## RAISE\_ERROR\_ERRNO

```
#define RAISE_ERROR_ERRNO(
    errtype,
    errmsg )
```

### Value:

```
{
    \
    fprintf(stderr, "%s:%d %s: %s (%s)\n",      \
        __FILE__, __LINE__,                    \
        __func__, (errmsg), strerror(errno));  \
    (errtype);                                \
}
```

## WRAP\_CHECK

```
#define WRAP_CHECK(
    errcode,
    jmpto,
    prev_errcode )
```

### Value:

```
if (prev_errcode != ERR_OK) {
    errcode = prev_errcode;
    goto jmp;to;
}
```

Wrapping function for native errors

## WRAP\_HDF5\_CHECK

```
#define WRAP_HDF5_CHECK(
    errcode,
    jmpto,
    result )
```

### Value:

```
{ if (result < 0) {
    errcode = ERR_DATASTREAM_WRITE;
    goto jmp;to;
} (result); }
```

Wrapping function for HDF5 errors

## 6.6 include/finds/evaluation.h File Reference

Derivative timing.

## Functions

- double `time_derivative_computation` (const `fish_system_t` \*system, const `derivative_computation_opts_t` dc\_opts)

*Times the computation of a single derivative.*

### 6.6.1 Detailed Description

Derivative timing.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.6.2 Function Documentation

#### `time_derivative_computation()`

```
double time_derivative_computation (
    const fish_system_t * system,
    const derivative_computation_opts_t dc_opts )
```

Times the computation of a single derivative.

#### Parameters

|    |                      |                                          |
|----|----------------------|------------------------------------------|
| in | <code>system</code>  | The system to compute the derivative of. |
| in | <code>dc_opts</code> | The derivative computation options.      |

#### Returns

The runtime required to compute the derivative.

## 6.7 include/finds/finds.h File Reference

```
#include "barneshut.h"
#include "constants.h"
```

```
#include "datastream.h"
#include "derivative.h"
#include "error.h"
#include "evaluation.h"
#include "finds.h"
#include "integration.h"
#include "interaction.h"
#include "simulation.h"
#include "system.h"
#include "util.h"
#include "vector.h"
```

## 6.8 include/finds/integration.h File Reference

Implementation of various adaptive and non-adaptive Runge-Kutta integrators.

```
#include <stdbool.h>
#include "system.h"
#include "derivative.h"
```

### Classes

- struct **integration\_opts\_t**  
*Struct for integration options.*

### Macros

- #define **DECLARE\_INTEGRATOR**(name)  
*Simple macro for declaring integrators (as all use the same args)*

### Typedefs

- typedef **fish\_system\_t** (\*)(**integration\_step\_fn**) (const **fish\_system\_t** \*state, const double time\_step, const **derivative\_computation\_opts\_t** dc\_opts, double \*error)

### Enumerations

- enum **integration\_method\_e** {  
    **EULER** , **RUNGE\_KUTTA\_2** , **RUNGE\_KUTTA\_4** , **RUNGE\_KUTTA\_5** ,  
    **RUNGE\_KUTTA\_6** , **RUNGE\_KUTTA\_32** , **RUNGE\_KUTTA\_54** , **INTEGRATION\_↵**  
    **METHODS\_COUNT** }  
*Enum for integration methods.*

## Functions

- **DECLARE\_INTEGRATOR** ([step\\_euler](#))
- **DECLARE\_INTEGRATOR** ([step\\_rk2](#))
- **DECLARE\_INTEGRATOR** ([step\\_rk4](#))
- **DECLARE\_INTEGRATOR** ([step\\_rk5](#))
- **DECLARE\_INTEGRATOR** ([step\\_rk6](#))
- **DECLARE\_INTEGRATOR** ([step\\_rk32](#))
- **DECLARE\_INTEGRATOR** ([step\\_rk54](#))

## Variables

- const [named\\_enum\\_t](#) **INTEGRATION\_METHODS\_TABLE** [INTEGRATION\_METHODS↵  
\_COUNT]

### 6.8.1 Detailed Description

Implementation of various adaptive and non-adaptive Runge-Kutta integrators.

This file contains the core routine for computing a singular integral time-step of the fish system (including optional error estimation).

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.8.2 Macro Definition Documentation

#### DECLARE\_INTEGRATOR

```
#define DECLARE_INTEGRATOR(  
    name )
```

#### Value:

```
fish_system_t *name( \  
    const fish_system_t *state, \  
    const double time_step, \  
    const derivative_computation_opts_t dc_opts, \  
    double *error \  
)
```

Simple macro for declaring integrators (as all use the same args)

### 6.8.3 Variable Documentation

#### INTEGRATION\_METHODS\_TABLE

```
const named_enum_t INTEGRATION_METHODS_TABLE [INTEGRATION_METHODS_COUNT]
[extern]
```

Named lookup table for integration methods

## 6.9 include/finds/interaction.h File Reference

Computation function for calculating the Laplacian interaction between two fish or two features.

```
#include <math.h>
#include "constants.h"
#include "vector.h"
#include "util.h"
#include "system.h"
```

### Typedefs

- typedef **double\_3d\_t**(\* **interaction\_fn\_t**) (const **double\_3d\_t** feat\_a\_pos, const **double\_3d\_t** feat\_b\_pos, const double regularization\_epsilon)

### 6.9.1 Detailed Description

Computation function for calculating the Laplacian interaction between two fish or two features.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

## 6.10 include/finds/simulation.h File Reference

Core simulation function.

```
#include <stdbool.h>
#include "integration.h"
#include "error.h"
```

## Functions

- **error\_e perform\_simulation** (const **fish\_system\_t** \*initial\_state, const **derivative\_computation\_opts\_t** dc\_opts, const **integration\_opts\_t** int\_opts, char \*output\_filename, char \*output\_folder\_name, const size\_t output\_filename\_size, const size\_t output\_folder\_size, const bool print\_file\_output)

*Performs a simulation using the given initial conditions.*

### 6.10.1 Detailed Description

Core simulation function.

This file contains the routine for performing a simulation over a specified integration time domain.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.10.2 Function Documentation

#### perform\_simulation()

```
error_e perform_simulation (
    const fish_system_t * initial_state,
    const derivative_computation_opts_t dc_opts,
    const integration_opts_t int_opts,
    char * output_filename,
    char * output_folder_name,
    const size_t output_filename_size,
    const size_t output_folder_size,
    const bool print_file_output )
```

Performs a simulation using the given initial conditions.

Runs the simulation based on the initial conditions and calculation parameters. Outputs to a file and produces both the filepath and the parent folder filepath.

#### Parameters

|     |                             |                                             |
|-----|-----------------------------|---------------------------------------------|
| in  | <i>initial_state</i>        | The initial state of the system.            |
| in  | <i>dc_opts</i>              | The derivative computation options.         |
| in  | <i>int_opts</i>             | The integral computation options.           |
| out | <i>output_filename</i>      | The output filename buffer for the dataset. |
| out | <i>output_folder_name</i>   | The output folder buffer.                   |
| in  | <i>output_filename_size</i> | The size of the output filename buffer.     |

## Returns

The error code for the simulation.

## 6.11 include/finds/system.h File Reference

Core particle system data structures and operations.

```
#include <string.h>
#include <stdbool.h>
#include <stdint.h>
#include <stddef.h>
#include "vector.h"
#include "util.h"
```

## Classes

- struct `distribution_options_t`  
*Distribution generation options struct.*
- struct `orientation_options_t`  
*Orientation generation options struct.*
- struct `constant_options_t`  
*Constant generation options struct.*
- struct `swimmer_t`  
*Structure representing an individual swimmer.*
- struct `fish_system_t`  
*Struct for a system of swimmers (i.e., a vector of swimmers).*
- struct `swimmer_features_t`  
*Structure of feature (source and sink) positions.*
- struct `feature_positions_t`  
*Vector of feature positions.*

## Enumerations

- enum `distribution_type_e` {  
DISTRIBUTION\_CUBE , DISTRIBUTION\_SPHERE , DISTRIBUTION\_BALL , DISTRIBUTION\_↵  
RANDOM ,  
DISTRIBUTION\_TYPE\_COUNT }  
*Enum for distribution generation types.*
- enum `orientation_type_e` {  
ORIENTATION\_RANDOM , ORIENTATION\_RADIAL\_INWARD , ORIENTATION\_↵  
RADIAL\_OUTWARD , ORIENTATION\_SWIRL\_INWARD ,  
ORIENTATION\_SWIRL\_OUTWARD , ORIENTATION\_SADDLE , ORIENTATION\_↵  
ALIGNED , ORIENTATION\_TYPE\_COUNT }  
*Enum for orientation generation types.*

## Functions

- `fish_system_t * fish_system_generate` (const `distribution_options_t` dist\_opts, const `orientation_options_t` ori\_opts, const `constant_options_t` const\_opts, const bool print\_↵ debug)  
*Generates a system of fish (positions and orientations).*
- `fish_system_t * fish_system_combine` (const `fish_system_t` \*a, const `fish_system_t` \*b)  
*Copies two systems to a greater system containing both.*
- `fish_system_t * fish_system_combine_destroy` (`fish_system_t` \*a, `fish_system_t` \*b)  
*Combines two fish systems and then destroys the previous systems.*
- `fish_system_t * fish_system_difference` (const `fish_system_t` \*a, const `fish_system_t` \*b)  
*Computes a fish-system representation of the difference.*
- `double fish_system_norm` (const `fish_system_t` \*system)  
*Computes the norm of a fish system.*
- `void fish_system_normalize_orientation` (`fish_system_t` \*system)  
*Normalizes the orientations for the fish system.*
- `bool fish_system_has_nan` (const `fish_system_t` \*system)  
*Determines whether a system constains a NaN value (for debugging).*
- `fish_system_t * fish_system_generate_random` (const `size_t` N, const bool print\_debug)  
*Generates a random system with predefined distribution and orientation options (both random) as a function of N, the number of swimmers in the system.*
- `fish_system_t * fish_system_copy` (const `fish_system_t` \*system)  
*Produces a fresh copy of a fish system.*
- `fish_system_t * fish_system_allocate` (const `size_t` N)  
*Allocates all of the arrays for a fish system to size N.*
- `void fish_system_print` (const `fish_system_t` \*system)  
*Prints the system to stdout (for debugging).*
- `void fish_system_destroy` (`fish_system_t` \*\*system)  
*Destroys the fish system (deallocates all pointers then redirects to NULL).*
- `void fish_system_translate` (`fish_system_t` \*system, const double delta\_x, const double delta\_y, const double delta\_z)  
*Translates all of the positions in a fish system.*
- `void fish_system_rotate` (`fish_system_t` \*system, const double roll, const double pitch, const double yaw)  
*Rotates all of the positions in a fish system.*
- `void calculate_swimmer_features` (const `double_3d_t` position, const `double_3d_t` orientation, const double length, `double_3d_t` \*source\_position\_ptr, `double_3d_t` \*sink\_↵ position\_ptr)  
*Computes the feature positions for a singular swimmer.*
- `feature_positions_t * calculate_feature_positions` (const `fish_system_t` \*system)  
*Calculates the feature positions for a fish system.*
- `void feature_positions_print` (const `feature_positions_t` \*feat\_pos)



*Prints the feature positions to stdout.*

- `feature_positions_t * feature_positions_allocate` (const size\_t N)

*Allocates the feature positions array.*

- void `feature_positions_destroy` (`feature_positions_t **feat_pos`)

*De-allocates the feature positions structure.*

## Variables

- const `named_enum_t DISTRIBUTION_TYPE_TABLE` [DISTRIBUTION\_TYPE\_COUNT]

*Enum naming table for distribution generation types.*

- const `named_enum_t ORIENTATION_TYPE_TABLE` [ORIENTATION\_TYPE\_COUNT]

*Enum naming table for orientation generation types.*

### 6.11.1 Detailed Description

Core particle system data structures and operations.

This file contains routines for creating, initializing, updating, and destroying static systems of fish-particles.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.11.2 Function Documentation

#### `calculate_feature_positions()`

```
feature_positions_t * calculate_feature_positions (  
    const fish_system_t * system )
```

Calculates the feature positions for a fish system.

Feature positions are calculated from the position and orientation as

source position = position + length / 2 \* orientation  
sink position = position - length / 2 \* orientation

**Parameters**

|    |               |                           |
|----|---------------|---------------------------|
| in | <i>system</i> | The system to compute on. |
|----|---------------|---------------------------|

**Returns**

The positions of the sources and sinks.

**calculate\_swimmer\_features()**

```
void calculate_swimmer_features (
    const double_3d_t position,
    const double_3d_t orientation,
    const double length,
    double_3d_t * source_position_ptr,
    double_3d_t * sink_position_ptr )
```

Computes the feature positions for a singular swimmer.

**Parameters**

|     |                            |                                            |
|-----|----------------------------|--------------------------------------------|
| in  | <i>position</i>            | The position of the swimmer.               |
| in  | <i>orientation</i>         | The orientation of the swimmer.            |
| in  | <i>length</i>              | The length of the swimmer.                 |
| out | <i>source_position_ptr</i> | The output vector for the source position. |
| out | <i>sink_position_ptr</i>   | The output vector for the sink position.   |

**feature\_positions\_allocate()**

```
feature_positions_t * feature_positions_allocate (
    const size_t N )
```

Allocates the feature positions array.

**Parameters**

|    |            |                     |
|----|------------|---------------------|
| in | <i>The</i> | size of the system. |
|----|------------|---------------------|

**feature\_positions\_destroy()**

```
void feature_positions_destroy (
    feature_positions_t ** feat_pos_ptr )
```

De-allocates the feature positions structure.

**Parameters**

|     |                     |                                |
|-----|---------------------|--------------------------------|
| out | <i>feat_pos_ptr</i> | The feature positions pointer. |
|-----|---------------------|--------------------------------|

**feature\_positions\_print()**

```
void feature_positions_print (
    const feature_positions_t * feat_pos )
```

Prints the feature positions to stdout.

**Parameters**

|    |                 |                        |
|----|-----------------|------------------------|
| in | <i>feat_pos</i> | The feature positions. |
|----|-----------------|------------------------|

**fish\_system\_allocate()**

```
fish_system_t * fish_system_allocate (
    const size_t N )
```

Allocates all of the arrays for a fish system to size N.

**Parameters**

|    |          |                              |
|----|----------|------------------------------|
| in | <i>N</i> | The size of the fish system. |
|----|----------|------------------------------|

**Returns**

The new allocated system.

**fish\_system\_combine()**

```
fish_system_t * fish_system_combine (
```

```
const fish_system_t * a,  
const fish_system_t * b )
```

Copies two systems to a greater system containing both.

#### Parameters

|    |          |                         |
|----|----------|-------------------------|
| in | <i>a</i> | The first fish system.  |
| in | <i>b</i> | The second fish system. |

#### Returns

The combined system.

### fish\_system\_combine\_destroy()

```
fish_system_t * fish_system_combine_destroy (  
    fish_system_t * a,  
    fish_system_t * b )
```

Combines two fish systems and then destroys the previous systems.

#### Parameters

|    |          |                         |
|----|----------|-------------------------|
| in | <i>a</i> | The first fish system.  |
| in | <i>b</i> | The second fish system. |

#### Returns

The combined system.

### fish\_system\_copy()

```
fish_system_t * fish_system_copy (  
    const fish_system_t * system )
```

Produces a fresh copy of a fish system.

#### Parameters

|    |               |                          |
|----|---------------|--------------------------|
| in | <i>system</i> | The fish system to copy. |
|----|---------------|--------------------------|

### Returns

The copied system.

### fish\_system\_destroy()

```
void fish_system_destroy (
    fish_system_t ** system_ptr )
```

Destroys the fish system (deallocates all pointers then redirects to NULL).

### Parameters

|     |                   |                                |
|-----|-------------------|--------------------------------|
| out | <i>system_ptr</i> | The system pointer to destroy. |
|-----|-------------------|--------------------------------|

### fish\_system\_difference()

```
fish_system_t * fish_system_difference (
    const fish_system_t * a,
    const fish_system_t * b )
```

Computes a fish-system representation of the difference.

### Parameters

|    |          |                    |
|----|----------|--------------------|
| in | <i>a</i> | The first system.  |
| in | <i>b</i> | The second system. |

### Returns

The difference.

### fish\_system\_generate()

```
fish_system_t * fish_system_generate (
    const distribution_options_t dist_opts,
    const orientation_options_t ori_opts,
    const constant_options_t const_opts,
    const bool print_debug )
```

Generates a system of fish (positions and orientations).

**Parameters**

|    |                    |                                                      |
|----|--------------------|------------------------------------------------------|
| in | <i>dist_opts</i>   | Distribution options.                                |
| in | <i>ori_opts</i>    | Orientation options.                                 |
| in | <i>const_opts</i>  | Constant generation options.                         |
| in | <i>print_debug</i> | Whether to print out that this system was generated. |

**Returns**

The system.

**fish\_system\_generate\_random()**

```
fish_system_t * fish_system_generate_random (
    const size_t N,
    const bool print_debug )
```

Generates a random system with predefined distribution and orientation options (both random) as a function of N, the number of swimmers in the system.

**Parameters**

|    |          |                                     |
|----|----------|-------------------------------------|
| in | <i>N</i> | The number of swimmers to generate. |
|----|----------|-------------------------------------|

**fish\_system\_has\_nan()**

```
bool fish_system_has_nan (
    const fish_system_t * system )
```

Determines whether a system contains a NaN value (for debugging).

**Parameters**

|    |               |             |
|----|---------------|-------------|
| in | <i>system</i> | The system. |
|----|---------------|-------------|

**Returns**

Whether the system contains NaN.

**fish\_system\_norm()**

```
double fish_system_norm (
    const fish_system_t * system )
```

Computes the norm of a fish system.

The norm of a fish system is defined as the square root of the sums of the squared-norms of the positions and orientations for all of the fish in the system,

$$|\mathbf{X}|^2 = \sum_{i=1}^N \left( |\mathbf{x}_{c,i}|^2 + |\mathbf{n}_i|^2 \right).$$

**Parameters**

|    |               |                                    |
|----|---------------|------------------------------------|
| in | <i>system</i> | The system to compute the norm of. |
|----|---------------|------------------------------------|

**Returns**

The norm of the system.

**fish\_system\_normalize\_orientation()**

```
void fish_system_normalize_orientation (
    fish_system_t * system )
```

Normalizes the orientations for the fish system.

**Parameters**

|     |               |                               |
|-----|---------------|-------------------------------|
| out | <i>system</i> | The fish system to normalize. |
|-----|---------------|-------------------------------|

**fish\_system\_print()**

```
void fish_system_print (
    const fish_system_t * system )
```

Prints the system to stdout (for debugging).

**Parameters**

|    |               |                           |
|----|---------------|---------------------------|
| in | <i>system</i> | The fish system to print. |
|----|---------------|---------------------------|

**fish\_system\_rotate()**

```
void fish_system_rotate (
    fish_system_t * system,
    const double roll,
    const double pitch,
    const double yaw )
```

Rotates all of the positions in a fish system.

**Parameters**

|     |               |                            |
|-----|---------------|----------------------------|
| out | <i>system</i> | The system to rotate.      |
| in  | <i>roll</i>   | The x-axis rotation angle. |
| in  | <i>pitch</i>  | The y-axis rotation angle. |
| in  | <i>yaw</i>    | The z-axis rotation angle. |

**fish\_system\_translate()**

```
void fish_system_translate (
    fish_system_t * system,
    const double delta_x,
    const double delta_y,
    const double delta_z )
```

Translates all of the positions in a fish system.

**Parameters**

|     |                             |                          |
|-----|-----------------------------|--------------------------|
| out | <i>system</i>               | The system to translate. |
| in  | <i>delta</i> ↔<br><i>_x</i> | The x-displacement.      |
| in  | <i>delta</i> ↔<br><i>_y</i> | The y-displacement.      |
| in  | <i>delta</i> ↔<br><i>_z</i> | The z-displacement.      |



## 6.12 include/finds/util.h File Reference

Utility functions.

```
#include <string.h>
#include <stdlib.h>
#include <time.h>
```

### Classes

- struct **named\_enum\_t**  
*Enum to name string mapping struct.*

### Macros

- #define **MAX**(a, b) ((a) > (b) ? (a) : (b))  
*Returns the max of a and b.*
- #define **MIN**(a, b) ((a) < (b) ? (a) : (b))  
*Returns the min of a and b.*
- #define **CLAMP**(val, min, max) (**MAX**((min), **MIN**((val), (max))))  
*Clamps a value between [min,max].*
- #define **ABOUT\_ZERO** 1E-20
- #define **ABS**(a) ((a) > 0 ? (a) : -(a))  
*Absolute value macro.*
- #define **IS\_CLOSE**(a, b) (**ABS**((a) - (b)) < **ABOUT\_ZERO**)  
*Comparison macro for comparing floating-point doubles.*
- #define **UNUSED** \_\_attribute\_\_((unused))  
*Shortened alias for unused parameters.*
- #define **NOT\_IMPLEMENTED**()  
*Macro for flagging functions that have not been implemented yet.*

### 6.12.1 Detailed Description

Utility functions.

This file contains routines misc. routines for various computations.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

## 6.12.2 Macro Definition Documentation

### ABOUT\_ZERO

```
#define ABOUT_ZERO 1E-20
```

About zero (with regard to the precision of this system).

### CLAMP

```
#define CLAMP(  
    val,  
    min,  
    max ) (MAX((min), MIN((val), (max))))
```

Clamps a value between [min,max].

#### Parameters

|    |            |                     |
|----|------------|---------------------|
| in | <i>val</i> | The value to clamp. |
| in | <i>min</i> | The minimum value.  |
| in | <i>max</i> | The maximum value.  |

#### Returns

If  $\text{min} < \text{val} < \text{min}$ , then *val*. If  $\text{val} < \text{min}$ , then *min*. If  $\text{val} > \text{max}$ , then *max*.

### NOT\_IMPLEMENTED

```
#define NOT_IMPLEMENTED( )
```

#### Value:

```
do { \  
    fprintf(stderr, "Function not implemented: %s\n", __func__); \  
    abort(); \  
} while (0)
```

Macro for flagging functions that have not been implemented yet.

## 6.13 include/finds/vector.h File Reference

Basic vector and matrix computations.

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <stdbool.h>
#include "util.h"
```

### Classes

- union `double_3d_t`  
*3-dimensional cartesian vector implementation*
- struct `mat3_t`  
*3x3 matrix implementation*

### Macros

- #define `D3(x, y, z)` ((`double_3d_t`) {x,y,z})  
*Shorthand for making D3 vectors.*
- #define `D3_FILL(a)` `D3(a,a,a)`  
*Fills a D3 with one value on each dimension.*
- #define `D3_ONE` `D3_FILL(1)`  
*A D3 of only 1s (1,1,1).*
- #define `D3_ZERO` `D3_FILL(0)`  
*A D3 of only 0s (0,0,0)*
- #define `MAT3(m11, m12, m13, m21, m22, m23, m31, m32, m33)`  
*Shorthand for creating 3x3 matrices.*

#### 6.13.1 Detailed Description

Basic vector and matrix computations.

This file contains routines for operating on 3-dimensional vectors and 3-by-3 matrices. It's focused mostly on speed, and as such, is incredibly small.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.13.2 Macro Definition Documentation

#### MAT3

```
#define MAT3(  
    m11,  
    m12,  
    m13,  
    m21,  
    m22,  
    m23,  
    m31,  
    m32,  
    m33 )
```

#### Value:

```
((mat3_t){ .data = {  
    {m11, m12, m13},  
    {m21, m22, m23},  
    {m31, m32, m33}  
}})
```

Shorthand for creating 3x3 matrices.

## 6.14 programs/benchmark.c File Reference

Produces a benchmarking-plot for the entirety of FINDS.

```
#include <math.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <gsl/gsl_fit.h>  
#include <gnuplot_i/gnuplot_i.h>  
#include <finds/system.h>  
#include <finds/datastream.h>  
#include <finds/constants.h>  
#include <finds/derivative.h>  
#include <finds/util.h>  
#include <finds/evaluation.h>
```

## Macros

- `#define PDF`
- `#define OUTPUT_MODE "pdf"`
- `#define FIGURE_WIDTH 3.5`
- `#define FIGURE_HEIGHT 2.9`
- `#define FONT "Nimbus Roman,11"`
- `#define SERIES_LABEL_SIZE 200`
- `#define SHOW_LEGEND_EXTENDED_DETAILS`
- `#define BRUTE_MAX_LOG_N 4`
- `#define BH_MAX_LOG_N 7`
- `#define FMM_MAX_LOG_N 8`
- `#define MIN_LOG_N 1`
- `#define MAX_LOG_N 6`
- `#define N_METHODS 9`

## Functions

- `int main (void)`

### 6.14.1 Detailed Description

Produces a benchmarking-plot for the entirety of FINDS.

Times the computation of a derivative for variable-N systems for each type of interaction computation method (brute force, Barnes-Hut at increasing theta, and the Fast-Multipole-Method at increasing order).

#### Author

Mufaro Machaya

License: MIT

## 6.15 programs/benchmark\_derivative.c File Reference

Times the computation of a single derivative.

```
#include <argp.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <finds/system.h>
#include <finds/derivative.h>
#include <finds/util.h>
```

## Classes

- struct `benchmark_options_t`

## Macros

- `#define DEFAULT_SIZE 100`
- `#define DEFAULT_METHOD BRUTE_FORCE`
- `#define DEFAULT_APPROX_THRESHOLD 0.0`
- `#define DEFAULT_PRECISION 1E-6`

## Functions

- int `main` (int argc, char \*argv[ ])

## Variables

- const char \* `argp_program_version` = "FINDS derivative timer v1.0"
- const char \* `argp_program_bug_address`

### 6.15.1 Detailed Description

Times the computation of a single derivative.

#### Author

Mufaro Machaya

License: MIT

### 6.15.2 Variable Documentation

#### `argp_program_bug_address`

```
const char* argp_program_bug_address
```

#### Initial value:

```
=
```

```
"Mufaro J. Machaya <mufaro2@student.ubc.ca>"
```

## 6.16 programs/linker\_test.c File Reference

Just a linker test that includes all of the libraries used in FINDS.

```
#include <argp.h>
#include <math.h>
#include <time.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <error.h>
#include <CL/cl.h>
#include <gsl/gsl_fit.h>
#include <gnuplot_i/gnuplot_i.h>
#include <progressbar/progressbar.h>
#include <tomlc17.h>
#include <hdf5.h>
#include <lfmm3d_c.h>
#include <finds/constants.h>
#include <finds/datastream.h>
#include <finds/derivative.h>
#include <finds/evaluation.h>
#include <finds/integration.h>
#include <finds/simulation.h>
#include <finds/system.h>
#include <finds/util.h>
#include <finds/vector.h>
```

### Functions

- `int main (void)`

#### 6.16.1 Detailed Description

Just a linker test that includes all of the libraries used in FINDS.

#### Author

Mufaro Machaya

License: MIT

## 6.17 programs/simulate.c File Reference

The core entrypoint for performing a simulation.

```
#include <argp.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <tomlc17.h>
#include <error.h>
#include <finds/system.h>
#include <finds/derivative.h>
#include <finds/simulation.h>
#include <finds/integration.h>
#include <finds/util.h>
#include <finds/constants.h>
```

### Classes

- struct `cmdline_args_t`
- struct `finds_opts_t`

### Macros

- #define `DEFAULT_SIMULATION_FILE` "default-sim-config.toml"
- #define `BUFFER_SIZE` 2048

### Functions

- int `main` (int argc, char \*argv[])

### Variables

- const char \* `argp_program_version` = "FINDS simulation v1.0"
- const char \* `argp_program_bug_address`



### 6.17.1 Detailed Description

The core entrypoint for performing a simulation.

Performs a simulation based on a user-supplied configuration file, then produces an output trajectory dataset in HDF5 format alongside a copy of the user-supplied configuration TOML.

#### Author

Mufaro Machaya

License: MIT

### 6.17.2 Variable Documentation

#### **argp\_program\_bug\_address**

```
const char* argp_program_bug_address
```

#### Initial value:

```
=  
    "Mufaro J. Machaya <mufaro2@student.ubc.ca>"
```

## 6.18 programs/validation.c File Reference

Produces the validation plots for FINDS.

```
#include <stdio.h>  
#include <stdlib.h>  
#include <math.h>  
#include <string.h>  
#include <float.h>  
#include <gnuplot_i/gnuplot_i.h>  
#include <finds/constants.h>  
#include <finds/system.h>  
#include <finds/simulation.h>  
#include <finds/error.h>  
#include <finds/datastream.h>
```

## Classes

- struct `system_trajectory_t`

*Swimmer trajectory object for simplifying the coplanar cases.*

## Macros

- `#define VALIDATION_OUTPUT_FOLDER` ROOT\_OUTPUT\_PATH "/validation"
- `#define TEMP_OUTPUT_FILE` VALIDATION\_OUTPUT\_FOLDER "/validation-circle-output.dat"
- `#define LENGTH` 1
- `#define VOLUMETRIC_FLOW_RATE` (4 \* M\_PI)
- `#define SP_SPEED` (VOLUMETRIC\_FLOW\_RATE / (4 \* M\_PI \* LENGTH \* LENGTH))
- `#define BUFFER_SIZE` 1024
- `#define DOUBLE_BUFFER_SIZE` (2 \* BUFFER\_SIZE)
- `#define PDF`
- `#define OUTPUT_MODE` "pdf"
- `#define LINEWIDTH` 1
- `#define FIGURE_WIDTH` 3.5
- `#define FIGURE_HEIGHT` 4
- `#define FONT` "Nimbus Roman,11"
- `#define N_DC_METHODS` 5
- `#define __WRAP_ERR`(errcode, fn)

## Functions

- `fish_system_t * comparison_plot_load_initial_conditions` (FILE \*restrict fptr, const size\_t N)
- `int main` (void)

### 6.18.1 Detailed Description

Produces the validation plots for FINDS.

Reproduces figures 8 and 16 of Mabrouk and Floryan 2025 and 2024 respectively using FINDS and GNUPlot.

#### Author

Mufaro Machaya

License: MIT

## 6.18.2 Macro Definition Documentation

### \_\_WRAP\_ERR

```
#define __WRAP_ERR(
    errcode,
    fn )
```

#### Value:

```
errcode = fn;
if (errcode != ERR_OK)
    return EXIT_FAILURE;
```

## 6.19 src/barneshut.c File Reference

External velocity contribution calculation through Barnes-Hut.

```
#include <assert.h>
#include <stdlib.h>
#include <stdbool.h>
#include <finds/barneshut.h>
#include <finds/system.h>
#include <finds/vector.h>
#include <finds/interaction.h>
```

## Classes

- struct **morton\_key\_t**  
*This maps morton codes to node indices in a linear octree.*
- struct **linear\_octree\_builder\_t**  
*A variable-sized wrapper for building linear octrees.*

## Macros

- #define **MORTON\_BITS** 21  
*The number of bits for storing morton codes (and subsequently, the maximum number of levels for the octree).*
- #define **LINEAR\_OCTREE\_REALLOC**(arrptr, cap) arrptr = realloc((arrptr), (cap) \* sizeof \*(arrptr))  
*Simple macro for reallocating the size of an octree (only for internal use when building).*
- #define **VECTOR\_CLUSTER\_APPEND**(sum, vec, w) ((sum) = d3\_add((sum), d3\_↔mult((vec), (w))))  
*Simple macro for adding vector data to a cluster sum.*

## Functions

- void `linear_octree_print` (const `linear_octree_t` \*restrict tree)  
*Prints a full octree to STDOUT (for debugging).*
- bool `linear_octree_check_nan` (const `linear_octree_t` \*restrict tree, bool verbose)  
*Checks if a linear octree has any NaN values.*
- `linear_octree_t` \* `linear_octree_build` (const `fish_system_t` \*restrict system)  
*Builds a linear octree from a fish system.*
- void `linear_octree_destroy` (`linear_octree_t` \*\*octree\_ptr)  
*De-allocates a linear octree.*
- void `linear_octree_compute_vel_contrib` (const `linear_octree_t` \*restrict tree, const size\_t swimmer\_i\_index, const `double_3d_t` swimmer\_i\_pos, const `swimmer_features_t` features\_i, const double approx\_ratio, const bool regularize, const double eps, `double_3d_t` \*restrict external\_source, `double_3d_t` \*restrict external\_sink)  
*Computes the velocity contribution on a swimmer by all of the non-self swimmers in the system.*

### 6.19.1 Detailed Description

External velocity contribution calculation through Barnes-Hut.

This file contains an implementation of the Barnes-Hut algorithm for external velocity contribution using linear octrees as a backend.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.19.2 Macro Definition Documentation

#### MORTON\_BITS

```
#define MORTON_BITS 21
```

The number of bits for storing morton codes (and subsequently, the maximum number of levels for the octree).

In essence, there are  $\text{floor}(64 \text{ bits} / 3 \text{ axes}) = 21$  bits available to each dimension (x, y, and z) using 64-bit integers, allowing our tree to have up to 21 levels (for a theoretical maximum of  $9.2E18$  nodes in the tree).

### 6.19.3 Function Documentation

#### **linear\_octree\_build()**

```
linear_octree_t * linear_octree_build (  
    const fish_system_t *restrict system )
```

Builds a linear octree from a fish system.

### Parameters

|    |               |                                    |
|----|---------------|------------------------------------|
| in | <i>system</i> | The fish system to construct from. |
|----|---------------|------------------------------------|

### Returns

The constructed linear octree.

### linear\_octree\_check\_nan()

```
bool linear_octree_check_nan (
    const linear_octree_t *restrict tree,
    bool verbose )
```

Checks if a linear octree has any NaN values.

### Parameters

|    |                |                                                      |
|----|----------------|------------------------------------------------------|
| in | <i>tree</i>    | The octree.                                          |
| in | <i>verbose</i> | Whether to print the exact value which contains NaN. |

### Returns

Whether or not any NaN values were found inside the tree.

### linear\_octree\_compute\_vel\_contrib()

```
void linear_octree_compute_vel_contrib (
    const linear_octree_t *restrict tree,
    const size_t swimmer_i_index,
    const double_3d_t swimmer_i_pos,
    const swimmer_features_t features_i,
    const double approx_ratio,
    const bool regularize,
    const double eps,
    double_3d_t *restrict external_source,
    double_3d_t *restrict external_sink )
```

Computes the velocity contribution on a swimmer by all of the non-self swimmers in the system.

**Parameters**

|     |                        |                                                |
|-----|------------------------|------------------------------------------------|
| in  | <i>tree</i>            | The linear octree.                             |
| in  | <i>swimmer_i_index</i> | The swimmer index for swimmer i.               |
| in  | <i>swimmer_i_pos</i>   | The position of swimmer i.                     |
| in  | <i>features_i</i>      | The feature positions of swimmer i.            |
| in  | <i>approx_ratio</i>    | The Barnes-Hut approximation ratio.            |
| in  | <i>regularize</i>      | Whether or not to use regularized interaction. |
| in  | <i>eps</i>             | Regularization Epsilon                         |
| out | <i>external_source</i> | The output vec. for external source contrib.   |
| out | <i>external_sink</i>   | The output vec. for external sink contrib.     |

**linear\_octree\_destroy()**

```
void linear_octree_destroy (
    linear_octree_t ** octree_ptr )
```

De-allocates a linear octree.

**Parameters**

|     |                   |                                     |
|-----|-------------------|-------------------------------------|
| out | <i>octree_ptr</i> | The point to the octree to destroy. |
|-----|-------------------|-------------------------------------|

**linear\_octree\_print()**

```
void linear_octree_print (
    const linear_octree_t *restrict tree )
```

Prints a full octree to STDOUT (for debugging).

**Parameters**

|    |             |                      |
|----|-------------|----------------------|
| in | <i>tree</i> | The octree to print. |
|----|-------------|----------------------|

**6.20 src/datastream.c File Reference**

I/O routines.

```
#include <errno.h>
#include <limits.h>
```

```
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <stdio.h>
#include <hdf5.h>
#include <finds/vector.h>
#include <finds/error.h>
#include <finds/datastream.h>
```

## Functions

- **error\_e mkdir\_p** (const char \*path, const mode\_t mode)  
*C implementation of "mkdir -p" command.*
- **error\_e datastream\_open\_file** (datastream\_t \*stream, const char \*filename)  
*Opens a dataset file for reading.*
- **error\_e datastream\_read\_frame** (datastream\_t \*stream, const size\_t frame, fish\_system\_t \*\*dest\_ptr, double \*time)
- **error\_e datastream\_create\_file** (datastream\_t \*stream, const char \*filename, const size\_t N)  
*Creates the root file for the datastream and marks it as open.*
- **error\_e datastream\_close** (datastream\_t \*stream)  
*Closes this datastream.*
- **error\_e datastream\_write\_system** (datastream\_t \*stream, const fish\_system\_t \*system, const double time)  
*Writes the state of a single system out to the datastream.*

### 6.20.1 Detailed Description

I/O routines.

This contains the implementation of the datastream object, which is used for creating and writing to simulation data files.

Author

Mufaro Machaya

License: MIT

### 6.20.2 Function Documentation

#### datastream\_close()

```
error_e datastream_close (
    datastream_t * stream )
```

Closes this datastream.



**Parameters**

|     |               |                      |
|-----|---------------|----------------------|
| out | <i>stream</i> | The stream to close. |
|-----|---------------|----------------------|

**Returns**

The error code for this process.

**datastream\_create\_file()**

```
error_e datastream_create_file (
    datastream_t * stream,
    const char * filename,
    const size_t N )
```

Creates the root file for the datastream and marks it as open.

**Parameters**

|     |                 |                                                  |
|-----|-----------------|--------------------------------------------------|
| out | <i>stream</i>   | The datastream to open.                          |
| in  | <i>filename</i> | The output filename for the dataset.             |
| in  | <i>N</i>        | The number of swimmers to track in this dataset. |

**Returns**

The error code for this process.

**datastream\_open\_file()**

```
error_e datastream_open_file (
    datastream_t * stream,
    const char * filename )
```

Opens a dataset file for reading.

**Parameters**

|     |                 |                            |
|-----|-----------------|----------------------------|
| out | <i>stream</i>   | The datastream to read to. |
| in  | <i>filename</i> | The file to open.          |

## Returns

The error code.

## datastream\_write\_system()

```
error_e datastream_write_system (
    datastream_t * stream,
    const fish_system_t * system,
    const double time )
```

Writes the state of a single system out to the datastream.

## Parameters

|     |               |                             |
|-----|---------------|-----------------------------|
| out | <i>stream</i> | The datastream to write to. |
| in  | <i>system</i> | The system state to write.  |
| in  | <i>time</i>   | The time marker to write.   |

## Returns

The error code for this operation.

## mkdir\_p()

```
error_e mkdir_p (
    const char * path,
    const mode_t mode )
```

C implementation of "mkdir -p" command.

Note: I'll be honest, I just stole this from generative AI. It works quite okay, though. Another possible implementation could just directly use mkdir -p via a system call (as this code doesn't need to be portable due to running in a virtual machine).

## Parameters

|    |             |                            |
|----|-------------|----------------------------|
| in | <i>path</i> | The folder path to create. |
| in | <i>mode</i> | The mkdir creation mode.   |

## Returns

The error code for this process.

## 6.21 src/evaluation.c File Reference

Performance evaluation.

```
#include <time.h>
#include <finds/system.h>
#include <finds/derivative.h>
#include <finds/util.h>
```

## Functions

- double `time_derivative_computation` (const `fish_system_t` \*system, const `derivative_computation_opts_t` dc\_opts)  
*Times the computation of a single derivative.*

### 6.21.1 Detailed Description

Performance evaluation.

This file contains the function implementation for timing the computation of a single derivative.

## Author

Mufaro J. Machaya

License: MIT

### 6.21.2 Function Documentation

#### `time_derivative_computation()`

```
double time_derivative_computation (
    const fish_system_t * system,
    const derivative_computation_opts_t dc_opts )
```

Times the computation of a single derivative.

## Parameters

|    |                |                                          |
|----|----------------|------------------------------------------|
| in | <i>system</i>  | The system to compute the derivative of. |
| in | <i>dc_opts</i> | The derivative computation options.      |

## Returns

The runtime required to compute the derivative.

## 6.22 src/integration.c File Reference

Integrator implementations.

```
#include <finds/system.h>
#include <finds/derivative.h>
#include <finds/integration.h>
#include <finds/error.h>
```

## Functions

- `fish_system_t * step_euler` (const `fish_system_t` \*state, const double time\_step, const `derivative_computation_opts_t` dc\_opts, double \*error)  
*Naive Euler-Stepping without Error Estimation.*
- `fish_system_t * step_rk2` (const `fish_system_t` \*state, const double time\_step, const `derivative_computation_opts_t` dc\_opts, double \*error)  
*Naive 2nd-order Runge-Kutta.*
- `fish_system_t * step_rk4` (const `fish_system_t` \*state, const double time\_step, const `derivative_computation_opts_t` dc\_opts, double \*error)  
*Naive 4th-order Runge-Kutta.*
- `fish_system_t * step_rk5` (const `fish_system_t` \*state, const double time\_step, const `derivative_computation_opts_t` dc\_opts, double \*error)  
*Naive 5th-order Runge-Kutta.*
- `fish_system_t * step_rk6` (const `fish_system_t` \*state, const double time\_step, const `derivative_computation_opts_t` dc\_opts, double \*error)  
*Naive 6th-order Runge-Kutta.*
- `fish_system_t * step_rk32` (const `fish_system_t` \*state, const double time\_step, const `derivative_computation_opts_t` dc\_opts, double \*error)  
*3rd-order Runge-Kutta (Bogacki-Shampine) with 2nd order error estimation.*
- `fish_system_t * step_rk54` (const `fish_system_t` \*state, const double time\_step, const `derivative_computation_opts_t` dc\_opts, double \*error)  
*5th-order Runge-Kutta (Dormand-Prince) with 4th order error estimation.*

## Variables

- const `named_enum_t` `INTEGRATION_METHODS_TABLE` [`INTEGRATION_METHODS`↵↵`_COUNT`]

### 6.22.1 Detailed Description

Integrator implementations.

This file contains the numerical integration schemes to be used in performing a simulation over some time domain, including forward Euler and embedded Runge-Kutta of variable order.

#### Author

Mufaro J. Machaya

License: MIT

### 6.22.2 Function Documentation

#### `step_euler()`

```
fish_system_t * step_euler (
    const fish_system_t * state,
    const double time_step,
    const derivative_computation_opts_t dc_opts,
    double * error )
```

Naive Euler-Stepping without Error Estimation.

#### Parameters

|     |                  |                                    |
|-----|------------------|------------------------------------|
| in  | <i>state</i>     | The system state for this time.    |
| in  | <i>time_step</i> | The time-step to advance to.       |
| out | <i>error</i>     | The RMS error-norm for this state. |

#### Returns

The advanced state  $y(t + \delta t)$ .

**step\_rk2()**

```
fish_system_t * step_rk2 (
    const fish_system_t * state,
    const double time_step,
    const derivative_computation_opts_t dc_opts,
    double * error )
```

Naive 2nd-order Runge-Kutta.

**Parameters**

|     |                  |                                    |
|-----|------------------|------------------------------------|
| in  | <i>state</i>     | The system state for this time.    |
| in  | <i>time_step</i> | The time-step to advance to.       |
| out | <i>error</i>     | The RMS error-norm for this state. |

**Returns**

The advanced state  $y(t + \delta t)$ .

**step\_rk32()**

```
fish_system_t * step_rk32 (
    const fish_system_t * state,
    const double time_step,
    const derivative_computation_opts_t dc_opts,
    double * error )
```

3rd-order Runge-Kutta (Bogacki-Shampine) with 2nd order error estimation.

**Parameters**

|     |                  |                                    |
|-----|------------------|------------------------------------|
| in  | <i>state</i>     | The system state for this time.    |
| in  | <i>time_step</i> | The time-step to advance to.       |
| out | <i>error</i>     | The RMS error-norm for this state. |

**Returns**

The advanced state  $y(t + \delta t)$ .

**step\_rk4()**

```
fish_system_t * step_rk4 (
    const fish_system_t * state,
    const double time_step,
    const derivative_computation_opts_t dc_opts,
    double * error )
```

Naive 4th-order Runge-Kutta.

**Parameters**

|     |                  |                                    |
|-----|------------------|------------------------------------|
| in  | <i>state</i>     | The system state for this time.    |
| in  | <i>time_step</i> | The time-step to advance to.       |
| out | <i>error</i>     | The RMS error-norm for this state. |

**Returns**

The advanced state  $y(t + \delta t)$ .

**step\_rk5()**

```
fish_system_t * step_rk5 (
    const fish_system_t * state,
    const double time_step,
    const derivative_computation_opts_t dc_opts,
    double * error )
```

Naive 5th-order Runge-Kutta.

**Parameters**

|     |                  |                                    |
|-----|------------------|------------------------------------|
| in  | <i>state</i>     | The system state for this time.    |
| in  | <i>time_step</i> | The time-step to advance to.       |
| out | <i>error</i>     | The RMS error-norm for this state. |

**Returns**

The advanced state  $y(t + \delta t)$ .

**step\_rk54()**

```
fish_system_t * step_rk54 (
    const fish_system_t * state,
    const double time_step,
    const derivative_computation_opts_t dc_opts,
    double * error )
```

5th-order Runge-Kutta (Dormand-Prince) with 4th order error estimation.

**Parameters**

|     |                  |                                    |
|-----|------------------|------------------------------------|
| in  | <i>state</i>     | The system state for this time.    |
| in  | <i>time_step</i> | The time-step to advance to.       |
| out | <i>error</i>     | The RMS error-norm for this state. |

**Returns**

The advanced state  $y(t + \text{delta } t)$ .

**step\_rk6()**

```
fish_system_t * step_rk6 (
    const fish_system_t * state,
    const double time_step,
    const derivative_computation_opts_t dc_opts,
    double * error )
```

Naive 6th-order Runge-Kutta.

**Parameters**

|     |                  |                                    |
|-----|------------------|------------------------------------|
| in  | <i>state</i>     | The system state for this time.    |
| in  | <i>time_step</i> | The time-step to advance to.       |
| out | <i>error</i>     | The RMS error-norm for this state. |



## Returns

The advanced state  $y(t + \delta t)$ .

### 6.22.3 Variable Documentation

#### INTEGRATION\_METHODS\_TABLE

```
const named_enum_t INTEGRATION_METHODS_TABLE [INTEGRATION_METHODS_COUNT]
```

#### Initial value:

```
= {  
    { EULER,          "euler" },  
    { RUNGE_KUTTA_2,  "RK2"   },  
    { RUNGE_KUTTA_4,  "RK4"   },  
    { RUNGE_KUTTA_5,  "RK5"   },  
    { RUNGE_KUTTA_6,  "RK6"   },  
  
    { RUNGE_KUTTA_32, "RK32"  },  
    { RUNGE_KUTTA_54, "RK54"  }  
}
```

Named lookup table for integration methods

## 6.23 src/simulation.c File Reference

Core simulation routine.

```
#include <math.h>  
#include <time.h>  
#include <stdio.h>  
#include <string.h>  
#include <stdlib.h>  
#include <progressbar/progressbar.h>  
#include <finds/util.h>  
#include <finds/integration.h>  
#include <finds/simulation.h>  
#include <finds/datastream.h>  
#include <finds/constants.h>
```

## Functions

- **error\_e perform\_simulation** (const **fish\_system\_t** \*initial\_state, const **derivative\_computation\_opts\_t** dc\_opts, const **integration\_opts\_t** int\_opts, char \*output\_filename, char \*output\_folder↔\_name, const size\_t output\_filename\_size, const size\_t output\_folder\_size, const bool print\_file\_output)

*Performs a simulation using the given initial conditions.*

### 6.23.1 Detailed Description

Core simulation routine.

This file contains the leading function for running a fresh simulation and producing the associated simulation datafile.

#### Author

Mufaro J. Machaya

License: MIT

### 6.23.2 Function Documentation

#### perform\_simulation()

```
error_e perform_simulation (
    const fish_system_t * initial_state,
    const derivative_computation_opts_t dc_opts,
    const integration_opts_t int_opts,
    char * output_filename,
    char * output_folder_name,
    const size_t output_filename_size,
    const size_t output_folder_size,
    const bool print_file_output )
```

Performs a simulation using the given initial conditions.

Runs the simulation based on the initial conditions and calculation parameters. Outputs to a file and produces both the filepath and the parent folder filepath.

#### Parameters

|     |                             |                                             |
|-----|-----------------------------|---------------------------------------------|
| in  | <i>initial_state</i>        | The initial state of the system.            |
| in  | <i>dc_opts</i>              | The derivative computation options.         |
| in  | <i>int_opts</i>             | The integral computation options.           |
| out | <i>output_filename</i>      | The output filename buffer for the dataset. |
| out | <i>output_folder_name</i>   | The output folder buffer.                   |
| in  | <i>output_filename_size</i> | The size of the output filename buffer.     |
| in  | <i>otuput_folder_size</i>   | The size of the output folder buffer.       |
| in  | <i>print_file_output</i>    | Whether to print the output filename.       |

## Returns

The error code for the simulation.

## 6.24 src/system.c File Reference

Core particle system data structures and operations.

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <finds/system.h>
#include <finds/util.h>
```

## Functions

- `fish_system_t * fish_system_generate` (const `distribution_options_t` dist\_opts, const `orientation_options_t` ori\_opts, const `constant_options_t` const\_opts, const bool print\_↔ debug)
 

*Generates a system of fish (positions and orientations).*
- `fish_system_t * fish_system_combine` (const `fish_system_t` \*a, const `fish_system_t` \*b)
 

*Copies two systems to a greater system containing both.*
- double `fish_system_norm` (const `fish_system_t` \*system)
 

*Computes the norm of a fish system.*
- `fish_system_t * fish_system_difference` (const `fish_system_t` \*a, const `fish_system_t` \*b)
 

*Computes a fish-system representation of the difference.*
- `fish_system_t * fish_system_combine_destroy` (`fish_system_t` \*a, `fish_system_t` \*b)
 

*Combines two fish systems and then destroys the previous systems.*
- void `fish_system_normalize_orientation` (`fish_system_t` \*system)
 

*Normalizes the orientations for the fish system.*
- void `fish_system_print` (const `fish_system_t` \*system)
 

*Prints the system to stdout (for debugging).*
- `fish_system_t * fish_system_copy` (const `fish_system_t` \*system)
 

*Produces a fresh copy of a fish system.*
- `fish_system_t * fish_system_generate_random` (const size\_t N, const bool print\_debug)
 

*Generates a random system with predefined distribution and orientation options (both random) as a function of N, the number of swimmers in the system.*
- bool `fish_system_has_nan` (const `fish_system_t` \*system)
 

*Determines whether a system contains a NaN value (for debugging).*
- `fish_system_t * fish_system_allocate` (const size\_t N)

*Allocates all of the arrays for a fish system to size N.*

- void `fish_system_destroy` (`fish_system_t` \*\*system\_ptr)

*Destroys the fish system (deallocates all pointers then redirects to NULL).*

- void `fish_system_translate` (`fish_system_t` \*system, const double delta\_x, const double delta\_y, const double delta\_z)

*Translates all of the positions in a fish system.*

- void `fish_system_rotate` (`fish_system_t` \*system, const double roll, const double pitch, const double yaw)

*Rotates all of the positions in a fish system.*

- void `calculate_swimmer_features` (const `double_3d_t` position, const `double_3d_t` orientation, const double length, `double_3d_t` \*source\_position\_ptr, `double_3d_t` \*sink\_↵ position\_ptr)

*Computes the feature positions for a singular swimmer.*

- `feature_positions_t` \* `calculate_feature_positions` (const `fish_system_t` \*system)

*Calculates the feature positions for a fish system.*

- void `feature_positions_print` (const `feature_positions_t` \*feat\_pos)

*Prints the feature positions to stdout.*

- `feature_positions_t` \* `feature_positions_allocate` (const size\_t N)

*Allocates the feature positions array.*

- void `feature_positions_destroy` (`feature_positions_t` \*\*feat\_pos\_ptr)

*De-allocates the feature positions structure.*

## Variables

- const `named_enum_t` `DISTRIBUTION_TYPE_TABLE` [`DISTRIBUTION_TYPE_COUNT`]

*Enum naming table for distribution generation types.*

- const `named_enum_t` `ORIENTATION_TYPE_TABLE` [`ORIENTATION_TYPE_COUNT`]

*Enum naming table for orientation generation types.*

### 6.24.1 Detailed Description

Core particle system data structures and operations.

This file contains routines for creating, initializing, updating, and destroying static systems of fish-particles.

#### Author

Mufaro J. Machaya

License: MIT

## 6.24.2 Function Documentation

### calculate\_feature\_positions()

```
feature_positions_t * calculate_feature_positions (
    const fish_system_t * system )
```

Calculates the feature positions for a fish system.

Feature positions are calculated from the position and orientation as

source position = position + length / 2 \* orientation  
sink position = position - length / 2 \* orientation

#### Parameters

|    |               |                           |
|----|---------------|---------------------------|
| in | <i>system</i> | The system to compute on. |
|----|---------------|---------------------------|

#### Returns

The positions of the sources and sinks.

### calculate\_swimmer\_features()

```
void calculate_swimmer_features (
    const double_3d_t position,
    const double_3d_t orientation,
    const double length,
    double_3d_t * source_position_ptr,
    double_3d_t * sink_position_ptr )
```

Computes the feature positions for a singular swimmer.

#### Parameters

|     |                            |                                            |
|-----|----------------------------|--------------------------------------------|
| in  | <i>position</i>            | The position of the swimmer.               |
| in  | <i>orientation</i>         | The orientation of the swimmer.            |
| in  | <i>length</i>              | The length of the swimmer.                 |
| out | <i>source_position_ptr</i> | The output vector for the source position. |
| out | <i>sink_position_ptr</i>   | The output vector for the sink position.   |

**feature\_positions\_allocate()**

```
feature_positions_t * feature_positions_allocate (
    const size_t N )
```

Allocates the feature positions array.

**Parameters**

|    |            |                     |
|----|------------|---------------------|
| in | <i>The</i> | size of the system. |
|----|------------|---------------------|

**feature\_positions\_destroy()**

```
void feature_positions_destroy (
    feature_positions_t ** feat_pos_ptr )
```

De-allocates the feature positions structure.

**Parameters**

|     |                     |                                |
|-----|---------------------|--------------------------------|
| out | <i>feat_pos_ptr</i> | The feature positions pointer. |
|-----|---------------------|--------------------------------|

**feature\_positions\_print()**

```
void feature_positions_print (
    const feature_positions_t * feat_pos )
```

Prints the feature positions to stdout.

**Parameters**

|    |                 |                        |
|----|-----------------|------------------------|
| in | <i>feat_pos</i> | The feature positions. |
|----|-----------------|------------------------|

**fish\_system\_allocate()**

```
fish_system_t * fish_system_allocate (
    const size_t N )
```

Allocates all of the arrays for a fish system to size N.

**Parameters**

|    |     |                              |
|----|-----|------------------------------|
| in | $N$ | The size of the fish system. |
|----|-----|------------------------------|

**Returns**

The new allocated system.

**fish\_system\_combine()**

```
fish_system_t * fish_system_combine (
    const fish_system_t * a,
    const fish_system_t * b )
```

Copies two systems to a greater system containing both.

**Parameters**

|    |     |                         |
|----|-----|-------------------------|
| in | $a$ | The first fish system.  |
| in | $b$ | The second fish system. |

**Returns**

The combined system.

**fish\_system\_combine\_destroy()**

```
fish_system_t * fish_system_combine_destroy (
    fish_system_t * a,
    fish_system_t * b )
```

Combines two fish systems and then destroys the previous systems.

**Parameters**

|    |     |                         |
|----|-----|-------------------------|
| in | $a$ | The first fish system.  |
| in | $b$ | The second fish system. |

### Returns

The combined system.

### fish\_system\_copy()

```
fish_system_t * fish_system_copy (
    const fish_system_t * system )
```

Produces a fresh copy of a fish system.

### Parameters

|    |               |                          |
|----|---------------|--------------------------|
| in | <i>system</i> | The fish system to copy. |
|----|---------------|--------------------------|

### Returns

The copied system.

### fish\_system\_destroy()

```
void fish_system_destroy (
    fish_system_t ** system_ptr )
```

Destroys the fish system (deallocates all pointers then redirects to NULL).

### Parameters

|     |                   |                                |
|-----|-------------------|--------------------------------|
| out | <i>system_ptr</i> | The system pointer to destroy. |
|-----|-------------------|--------------------------------|

### fish\_system\_difference()

```
fish_system_t * fish_system_difference (
    const fish_system_t * a,
    const fish_system_t * b )
```

Computes a fish-system representation of the difference.



**Parameters**

|    |          |                    |
|----|----------|--------------------|
| in | <i>a</i> | The first system.  |
| in | <i>b</i> | The second system. |

**Returns**

The difference.

**fish\_system\_generate()**

```
fish_system_t * fish_system_generate (
    const distribution_options_t dist_opts,
    const orientation_options_t ori_opts,
    const constant_options_t const_opts,
    const bool print_debug )
```

Generates a system of fish (positions and orientations).

**Parameters**

|    |                    |                                                      |
|----|--------------------|------------------------------------------------------|
| in | <i>dist_opts</i>   | Distribution options.                                |
| in | <i>ori_opts</i>    | Orientation options.                                 |
| in | <i>const_opts</i>  | Constant generation options.                         |
| in | <i>print_debug</i> | Whether to print out that this system was generated. |

**Returns**

The system.

**fish\_system\_generate\_random()**

```
fish_system_t * fish_system_generate_random (
    const size_t N,
    const bool print_debug )
```

Generates a random system with predefined distribution and orientation options (both random) as a function of N, the number of swimmers in the system.

**Parameters**

|    |          |                                     |
|----|----------|-------------------------------------|
| in | <i>N</i> | The number of swimmers to generate. |
|----|----------|-------------------------------------|

**fish\_system\_has\_nan()**

```
bool fish_system_has_nan (
    const fish_system_t * system )
```

Determines whether a system contains a NaN value (for debugging).

**Parameters**

|    |               |             |
|----|---------------|-------------|
| in | <i>system</i> | The system. |
|----|---------------|-------------|

**Returns**

Whether the system contains NaN.

**fish\_system\_norm()**

```
double fish_system_norm (
    const fish_system_t * system )
```

Computes the norm of a fish system.

The norm of a fish system is defined as the square root of the sums of the squared-norms of the positions and orientations for all of the fish in the system,

$$|\mathbf{X}|^2 = \sum_{i=1}^N \left( |\mathbf{x}_{c,i}|^2 + |\mathbf{n}_i|^2 \right).$$

**Parameters**

|    |               |                                    |
|----|---------------|------------------------------------|
| in | <i>system</i> | The system to compute the norm of. |
|----|---------------|------------------------------------|

### Returns

The norm of the system.

### fish\_system\_normalize\_orientation()

```
void fish_system_normalize_orientation (
    fish_system_t * system )
```

Normalizes the orientations for the fish system.

#### Parameters

|     |               |                               |
|-----|---------------|-------------------------------|
| out | <i>system</i> | The fish system to normalize. |
|-----|---------------|-------------------------------|

### fish\_system\_print()

```
void fish_system_print (
    const fish_system_t * system )
```

Prints the system to stdout (for debugging).

#### Parameters

|    |               |                           |
|----|---------------|---------------------------|
| in | <i>system</i> | The fish system to print. |
|----|---------------|---------------------------|

### fish\_system\_rotate()

```
void fish_system_rotate (
    fish_system_t * system,
    const double roll,
    const double pitch,
    const double yaw )
```

Rotates all of the positions in a fish system.

#### Parameters

|     |               |                            |
|-----|---------------|----------------------------|
| out | <i>system</i> | The system to rotate.      |
| in  | <i>roll</i>   | The x-axis rotation angle. |
| in  | <i>pitch</i>  | The y-axis rotation angle. |
| in  | <i>yaw</i>    | The z-axis rotation angle. |

**fish\_system\_translate()**

```
void fish_system_translate (
    fish_system_t * system,
    const double delta_x,
    const double delta_y,
    const double delta_z )
```

Translates all of the positions in a fish system.

**Parameters**

|     |                |                          |
|-----|----------------|--------------------------|
| out | <i>system</i>  | The system to translate. |
| in  | <i>delta_x</i> | The x-displacement.      |
| in  | <i>delta_y</i> | The y-displacement.      |
| in  | <i>delta_z</i> | The z-displacement.      |

**6.24.3 Variable Documentation****DISTRIBUTION\_TYPE\_TABLE**

```
const named_enum_t DISTRIBUTION_TYPE_TABLE[DISTRIBUTION_TYPE_COUNT]
```

**Initial value:**

```
= {
    { DISTRIBUTION_CUBE,      "cube" },
    { DISTRIBUTION_SPHERE,   "sphere" },
    { DISTRIBUTION_BALL,     "ball" },
    { DISTRIBUTION_RANDOM,   "random" }
}
```

Enum naming table for distribution generation types.

**ORIENTATION\_TYPE\_TABLE**

```
const named_enum_t ORIENTATION_TYPE_TABLE[ORIENTATION_TYPE_COUNT]
```

**Initial value:**

```
= {
    { ORIENTATION_RANDOM,      "random" },
    { ORIENTATION_RADIAL_INWARD, "radial-inward" },
    { ORIENTATION_RADIAL_OUTWARD, "radial-outward" },
    { ORIENTATION_SWIRL_INWARD, "swirl-inward" },
    { ORIENTATION_SWIRL_OUTWARD, "swirl-outward" },
    { ORIENTATION_SADDLE,      "saddle" },
    { ORIENTATION_ALIGNED,     "aligned" },
}
```

Enum naming table for orientation generation types.

## 6.25 tests/main.c File Reference

Entry-point for running unit tests.

```
#include <stdio.h>
#include <stdlib.h>
#include <check.h>
#include "tests.h"
```

### Functions

- `int main (void)`

#### 6.25.1 Detailed Description

Entry-point for running unit tests.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

## 6.26 tests/test\_datastream.c File Reference

Unit tests for the Datastream I/O module.

```
#include "tests.h"
#include <check.h>
#include <finds/datastream.h>
#include <finds/constants.h>
```

### Macros

- `#define SYSTEM_SIZE 100`
- `#define TEST_OUTPUT_FILE "test_datafile.h5"`
- `#define __WRAP(code) ck_assert(code == ERR_OK)`

## Functions

- **START\_TEST** (test\_read\_write\_hdf5)  
*Unit test for reading/writing datastream HDF5 files.*
- **END\_TEST** Suite \* **test\_suite\_datastream** (void)

### 6.26.1 Detailed Description

Unit tests for the Datastream I/O module.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

### 6.26.2 Function Documentation

#### START\_TEST()

```
START_TEST (
    test_read_write_hdf5 )
```

Unit test for reading/writing datastream HDF5 files.

1. The deserialized version of the data should be identical to the data before serialization.  
(We should be able to fully retrieve the data afterward.)

## 6.27 tests/test\_derivative.c File Reference

Unit tests for the derivative and interaction calculation module.

```
#include <stdlib.h>
#include <check.h>
#include <finds/derivative.h>
#include "tests.h"
```

## Functions

- **START\_TEST** (test\_compute\_derivative\_brute\_force)
- **END\_TEST START\_TEST** (test\_compute\_derivative\_barnes\_hut)
- **END\_TEST START\_TEST** (test\_compute\_derivative\_fast\_multipole\_method)
- **END\_TEST START\_TEST** (test\_regularization)
- **END\_TEST Suite \* test\_suite\_derivative** (void)

### 6.27.1 Detailed Description

Unit tests for the derivative and interaction calculation module.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

## 6.28 tests/test\_integration.c File Reference

Unit tests for the numeric integration module.

```
#include <check.h>
#include <finds/integration.h>
#include "tests.h"
```

## Functions

- **START\_TEST** (test\_step\_euler)
- **END\_TEST START\_TEST** (test\_step\_rk4)
- **END\_TEST Suite \* test\_suite\_integration** (void)

### 6.28.1 Detailed Description

Unit tests for the numeric integration module.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

## 6.29 tests/test\_octree.c File Reference

Unit tests for the linear octree generation.

```
#include <check.h>
#include <finds/barneshut.h>
#include "tests.h"
```

### Functions

- **START\_TEST** (test\_octree\_generation)
- **END\_TEST** Suite \* **test\_suite\_octree** (void)

#### 6.29.1 Detailed Description

Unit tests for the linear octree generation.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

## 6.30 tests/test\_simulation.c File Reference

Unit tests for the main simulation module.

```
#include <check.h>
#include <finds/simulation.h>
#include "tests.h"
```

### Functions

- **START\_TEST** (test\_simulation\_non\_adaptive\_rk4)
- **END\_TEST** **START\_TEST** (test\_simulation\_adaptive\_rk45)
- **END\_TEST** Suite \* **test\_suite\_simulation** (void)



### 6.30.1 Detailed Description

Unit tests for the main simulation module.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

## 6.31 tests/test\_system.c File Reference

Unit tests for the fish system generation module.

```
#include <check.h>
#include <finds/system.h>
#include "tests.h"
```

### Functions

- **START\_TEST** (test\_system\_random)
- **END\_TEST** Suite \* **test\_suite\_system** (void)

### 6.31.1 Detailed Description

Unit tests for the fish system generation module.

#### Author

Mufaro Machaya [mufaro2@student.ubc.ca](mailto:mufaro2@student.ubc.ca)

License: MIT

# Index

\_\_WRAP\_ERR  
validation.c, 78

ABOUT\_ZERO  
util.h, 69

append\_state  
processingmodules.AnimationGenerator.AnimationGenerator, 11  
processingmodules.CrossSectionGenerator.CrossSectionGenerator, 15  
processingmodules.DensityAnimationGenerator.DensityAnimationGenerator, 18  
processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot, 27  
processingmodules.ProcessingModule.ProcessingModule, 31  
processingmodules.SnapshotGenerator.SnapshotGenerator, 33  
processingmodules.TrajectoryPlot.TrajectoryPlot, 37

argp\_program\_bug\_address  
benchmark\_derivative.c, 73  
simulate.c, 76

barneshut.c  
linear\_octree\_build, 80  
linear\_octree\_check\_nan, 81  
linear\_octree\_compute\_vel\_contrib, 81  
linear\_octree\_destroy, 82  
linear\_octree\_print, 82  
MORTON\_BITS, 79

barneshut.h  
linear\_octree\_build, 40  
linear\_octree\_check\_nan, 40  
linear\_octree\_compute\_vel\_contrib, 41  
linear\_octree\_destroy, 41  
linear\_octree\_print, 42

begin  
processingmodules.AnimationGenerator.AnimationGenerator, 11  
processingmodules.CrossSectionGenerator.CrossSectionGenerator, 15  
processingmodules.DensityAnimationGenerator.DensityAnimationGenerator, 18  
processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot, 28  
processingmodules.ProcessingModule.ProcessingModule, 31  
processingmodules.SnapshotGenerator.SnapshotGenerator, 33  
processingmodules.TrajectoryPlot.TrajectoryPlot, 37  
benchmark\_derivative.c, 73  
argp\_program\_bug\_address, 73  
benchmark\_options\_t, 12  
calculate\_system\_derivatives, 96  
system.c, 96  
calculate\_swimmer\_features, 96  
system.h, 61  
util.h, 69  
outline\_args\_t, 12  
compute\_system\_derivative, 48  
derivative.h, 48  
constant\_options\_t, 13

datastream.c  
datastream\_close, 83  
datastream\_create\_file, 84  
datastream\_open\_file, 84  
datastream\_write\_system, 85  
mkdir\_p, 85

datastream.h  
datastream\_close, 44  
datastream\_create\_file, 44  
datastream\_open\_file, 44  
datastream\_write\_system, 46  
mkdir\_p, 46

datastream\_close  
datastream.c, 83  
datastream.h, 44

datastream\_create\_file  
datastream.c, 84  
datastream.h, 44

datastream\_open\_file  
datastream.c, 84  
datastream.h, 44

datastream\_write\_system  
datastream.c, 85  
datastream.h, 46

- datastream.c, 85
- datastream.h, 46
- DECLARE\_INTEGRATOR
  - integration.h, 55
- derivative.h
  - compute\_system\_derivative, 48
  - derivative\_average, 49
  - derivative\_destroy, 49
  - derivative\_has\_nan, 49
  - derivative\_print, 50
- derivative\_average
  - derivative.h, 49
- derivative\_computation\_opts\_t, 19
- derivative\_destroy
  - derivative.h, 49
- derivative\_has\_nan
  - derivative.h, 49
- derivative\_print
  - derivative.h, 50
- derivative\_weight\_t, 20
- distribution\_options\_t, 20
- DISTRIBUTION\_TYPE\_TABLE
  - system.c, 103
- double\_3d\_t, 21
- end
  - processingmodules.AnimationGenerator.AnimationGenerator, 11
  - processingmodules.CrossSectionGenerator.CrossSectionGenerator, 15
  - processingmodules.DensityAnimationGenerator.DensityAnimationGenerator, 19
  - processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot, 28
  - processingmodules.ProcessingModule.ProcessingModule, 31
  - processingmodules.SnapshotGenerator.SnapshotGenerator, 34
  - processingmodules.TrajectoryPlot.TrajectoryPlot, 38
- error.h
  - GOTO\_IF\_NULL, 51
  - RAISE\_ERROR, 51
  - RAISE\_ERROR\_ERRNO, 51
  - WRAP\_CHECK, 52
  - WRAP\_HDF5\_CHECK, 52
- evaluation.c
  - time\_derivative\_computation, 86
- evaluation.h
  - time\_derivative\_computation, 53
- feature\_positions\_allocate
  - system.c, 96
  - system.h, 61
- feature\_positions\_destroy
  - system.c, 97
  - system.h, 61
- feature\_positions\_print
  - system.c, 97
  - system.h, 62
- feature\_positions\_t, 22
- feature\_velocity\_t, 22
- FINDS: The Fish Interaction and Dynamics Simulator, 2
- finds\_opts\_t, 23
- fish\_system\_allocate
  - system.c, 97
  - system.h, 62
- fish\_system\_combine
  - system.c, 98
  - system.h, 62
- fish\_system\_combine\_destroy
  - system.c, 98
  - system.h, 63
- fish\_system\_copy
  - system.c, 99
  - system.h, 63
- fish\_system\_destroy
  - system.c, 99
- fish\_system\_difference
  - system.c, 100
  - system.h, 64
- fish\_system\_generate
  - system.c, 100
  - system.h, 64
- fish\_system\_generate\_random
  - system.c, 100
  - system.h, 65
- fish\_system\_has\_nan
  - system.c, 101
  - system.h, 65
- fish\_system\_norm
  - system.c, 101
  - system.h, 65
- fish\_system\_normalize\_orientation
  - system.c, 102
  - system.h, 66

- fish\_system\_print
  - system.c, 102
  - system.h, 66
- fish\_system\_rotate
  - system.c, 102
  - system.h, 67
- fish\_system\_t, 23
- fish\_system\_translate
  - system.c, 103
  - system.h, 67
- GOTO\_IF\_NULL
  - error.h, 51
- include/finds/barneshut.h, 39
- include/finds/constants.h, 42
- include/finds/datastream.h, 43
- include/finds/derivative.h, 47
- include/finds/error.h, 50
- include/finds/evaluation.h, 52
- include/finds/finds.h, 53
- include/finds/integration.h, 54
- include/finds/interaction.h, 56
- include/finds/simulation.h, 56
- include/finds/system.h, 58
- include/finds/util.h, 68
- include/finds/vector.h, 70
- integration.c
  - INTEGRATION\_METHODS\_TABLE, 92
  - step\_euler, 88
  - step\_rk2, 88
  - step\_rk32, 89
  - step\_rk4, 89
  - step\_rk5, 90
  - step\_rk54, 90
  - step\_rk6, 91
- integration.h
  - DECLARE\_INTEGRATOR, 55
  - INTEGRATION\_METHODS\_TABLE, 56
- INTEGRATION\_METHODS\_TABLE
  - integration.c, 92
  - integration.h, 56
- integration\_opts\_t, 24
- linear\_octree\_build
  - barneshut.c, 80
  - barneshut.h, 40
- linear\_octree\_builder\_t, 24
- linear\_octree\_check\_nan
  - barneshut.c, 81
  - barneshut.h, 40
- linear\_octree\_compute\_vel\_contrib
  - barneshut.c, 81
  - barneshut.h, 41
- linear\_octree\_destroy
  - barneshut.c, 82
  - barneshut.h, 41
- linear\_octree\_print
  - barneshut.c, 82
  - barneshut.h, 42
- linear\_octree\_t, 25
- MAT3
  - vector.h, 71
- mat3\_t, 26
- mkdir\_p
  - datastream.c, 85
  - datastream.h, 46
- MORTON\_BITS
  - barneshut.c, 79
- morton\_key\_t, 29
- named\_enum\_t, 29
- NOT\_IMPLEMENTED
  - util.h, 69
- orientation\_options\_t, 30
- ORIENTATION\_TYPE\_TABLE
  - system.c, 103
- perform\_simulation
  - simulation.c, 93
  - simulation.h, 57
- processingmodules.AnimationGenerator.AnimationGenerator,
  - 9
  - append\_state, 11
  - begin, 11
  - end, 11
- processingmodules.CrossSectionGenerator.CrossSectionGen
  - 13
  - append\_state, 15
  - begin, 15
  - end, 15
- processingmodules.DensityAnimationGenerator.DensityAnima
  - 17
  - append\_state, 18
  - begin, 18
  - end, 19

processingmodules.MeanRadialDistancePlot.MeanRadialDistancePlot,  
     26  
     append\_state, 27  
     begin, 28  
     end, 28  
 processingmodules.ProcessingModule.ProcessingModule,  
     30  
     append\_state, 31  
     begin, 31  
     end, 31  
 processingmodules.SnapshotGenerator.SnapshotGenerator,  
     32  
     append\_state, 33  
     begin, 33  
     end, 34  
 processingmodules.TrajectoryPlot.TrajectoryPlot,  
     36  
     append\_state, 37  
     begin, 38  
     end, 38  
 programs/benchmark.c, 71  
 programs/benchmark\_derivative.c, 72  
 programs/linker\_test.c, 74  
 programs/simulate.c, 75  
 programs/validation.c, 76  
  
 RAISE\_ERROR  
     error.h, 51  
 RAISE\_ERROR\_ERRNO  
     error.h, 51  
  
 simulate.c  
     argp\_program\_bug\_address, 76  
 simulation.c  
     perform\_simulation, 93  
 simulation.h  
     perform\_simulation, 57  
 src/barneshut.c, 78  
 src/datastream.c, 82  
 src/evaluation.c, 86  
 src/integration.c, 87  
 src/simulation.c, 92  
 src/system.c, 94  
 START\_TEST  
     test\_datastream.c, 105  
 step\_euler  
     integration.c, 88  
 step\_rk2  
     integration.c, 88  
     integration.c, 89  
     step\_rk4  
         integration.c, 89  
     step\_rk5  
         integration.c, 90  
     step\_rk54  
         integration.c, 90  
     step\_rk6  
         integration.c, 91  
     swimmer\_t, 34  
     swimmer\_t, 35  
     system.c  
         calculate\_feature\_positions, 96  
         calculate\_swimmer\_features, 96  
         DISTRIBUTION\_TYPE\_TABLE, 103  
         feature\_positions\_allocate, 96  
         feature\_positions\_destroy, 97  
         feature\_positions\_print, 97  
         fish\_system\_allocate, 97  
         fish\_system\_combine, 98  
         fish\_system\_combine\_destroy, 98  
         fish\_system\_copy, 99  
         fish\_system\_destroy, 99  
         fish\_system\_difference, 99  
         fish\_system\_generate, 100  
         fish\_system\_generate\_random, 100  
         fish\_system\_has\_nan, 101  
         fish\_system\_norm, 101  
         fish\_system\_normalize\_orientation, 102  
         fish\_system\_print, 102  
         fish\_system\_rotate, 102  
         fish\_system\_translate, 103  
         ORIENTATION\_TYPE\_TABLE, 103  
     system.h  
         calculate\_feature\_positions, 60  
         calculate\_swimmer\_features, 61  
         feature\_positions\_allocate, 61  
         feature\_positions\_destroy, 61  
         feature\_positions\_print, 62  
         fish\_system\_allocate, 62  
         fish\_system\_combine, 62  
         fish\_system\_combine\_destroy, 63  
         fish\_system\_copy, 63  
         fish\_system\_destroy, 64  
         fish\_system\_difference, 64  
         fish\_system\_generate, 64  
         fish\_system\_generate\_random, 65

- fish\_system\_has\_nan, 65
- fish\_system\_norm, 65
- fish\_system\_normalize\_orientation, 66
- fish\_system\_print, 66
- fish\_system\_rotate, 67
- fish\_system\_translate, 67
- system\_derivative\_t, 35
- system\_trajectory\_t, 36
- test\_datastream.c
  - START\_TEST, 105
- tests/main.c, 104
- tests/test\_datastream.c, 104
- tests/test\_derivative.c, 105
- tests/test\_integration.c, 106
- tests/test\_octree.c, 107
- tests/test\_simulation.c, 107
- tests/test\_system.c, 108
- time\_derivative\_computation
  - evaluation.c, 86
  - evaluation.h, 53
- util.h
  - ABOUT\_ZERO, 69
  - CLAMP, 69
  - NOT\_IMPLEMENTED, 69
- validation.c
  - \_\_WRAP\_ERR, 78
- vector.h
  - MAT3, 71
- WRAP\_CHECK
  - error.h, 52
- WRAP\_HDF5\_CHECK
  - error.h, 52